

**Konzeption und Implementierung eines KI-gestützten
GitLab-Assistenten zur Aufbereitung von Software-Tickets**

Bachelorarbeit

des Studienganges **Informatik**

an der Dualen Hochschule Baden-Württemberg **Stuttgart**

von

Nico Werner

—

Bearbeitungszeitraum	12 Wochen
Matrikelnummer, Kurs	3043145, INF23E
Dualer Partner	Drees & Sommer, Stuttgart
Betreuer*in des Dualen Partners	Denis Günther
Wissenschaftliche Betreuung	—

Ehrenwörtliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Ort, Datum

Unterschrift

Kurzfassung

Software-Tickets verbinden in Entwicklungsteams fachlichen Bedarf, Umsetzung und Prüfung. Relevante Angaben sind häufig unvollständig. Oft stehen sie verteilt in Beschreibung und Kommentaren. Diese Arbeit untersucht zwei Fragen: Welche Informationen braucht ein GitLab-Ticket, damit Umsetzung und Prüfung nachvollziehbar sind? Und wie lässt sich ein KI-gestützter Assistent kontrolliert in einen GitLab-Ablauf einbinden? Den fachlichen Kern bilden Ziel, Kontext, erwartetes Verhalten, Prüfkriterien und sichtbare Unsicherheit.

Dazu wird Dev-Assist als Machbarkeitsnachweis in TypeScript mit Express umgesetzt. GitLab-Webhooks lösen die Verarbeitung aus. Der Dienst sammelt Issue- und Kommentardaten sowie optionalen Repository-Kontext. Er übergibt sie an einen OpenCode-Analyseagenten und prüft die Antwort gegen ein festes Ausgabeschema. Je nach Informationslage entstehen Rückfragen oder ein strukturierter Vorschlag. Das Issue wird erst nach einem gesonderten Publish-Kommando geändert. Ausgabeschema, Schutz vor Bot-Schleifen und lokale Speicherung begrenzen, was das Modell selbst entscheiden darf.

Abstract

Software tickets are the interface between business needs, implementation, and verification in development teams. Relevant information is often incomplete or spread across the description and later comments. This thesis investigates which information a GitLab ticket needs for comprehensible implementation and checking, and how an AI assistant can be integrated into a GitLab workflow under explicit control. Goal, context, expected behaviour, acceptance criteria, and visible uncertainty form the quality core.

Dev-Assist is implemented as a TypeScript and Express proof of concept. GitLab webhooks trigger processing. The service collects issue and comment data plus optional repository context, sends them to an OpenCode analysis agent, and validates the reply against a fixed data contract. Depending on the information available, it asks clarification questions or produces a structured proposal. The issue is changed only after a separate publish command. The data contract, bot-loop protection, and local persistence limit the model's autonomy.

Inhaltsverzeichnis

Ehrenwörtliche Erklärung	I
Kurzfassung	II
Abstract	III
Abbildungsverzeichnis	VII
Tabellenverzeichnis	VIII
Abkürzungsverzeichnis	IX
1. Einleitung	1
1.1. Ziel und Vorgehen	2
1.2. Aufbau und Abgrenzung	2
2. Theoretische und technische Grundlagen	4
2.1. Requirements Engineering und Ticketqualität	4
2.1.1. Empirische Befunde zu User Stories und Bug Reports	4
2.1.2. Ableitung des Qualitätsmodells	5
2.2. Large Language Models und agentenbasierte Assistenz	6
2.3. Automatisierte Unterstützung und aktuelle LLM-Forschung	7
2.3.1. Regelbasierte und hybride Werkzeuge	7
2.3.2. Empirische Arbeiten zu LLMs im Requirements Engineering . .	8
2.4. GitLab als Arbeits- und Integrationsplattform	11
2.4.1. Entwicklungsplattform	11
2.4.2. Issues und Kommentarverlauf	11
2.4.3. Webhooks als Integrationspunkt	11
2.5. Risiken generativer Assistenz	12
2.6. Zwischenfazit	13
3. Anforderungsanalyse	14
3.1. Ausgangs- und Zielprozess	14
3.2. Stakeholder und Priorisierung	14
3.3. Qualitätsmodell für Ticketvorschläge	15
3.4. Funktionale Anforderungen	16
3.5. Nicht-funktionale Anforderungen	17
3.6. Herkunft und Nachverfolgbarkeit	18
3.7. Abgrenzung und Zwischenfazit	19

4. Konzeption der Systemarchitektur	21
4.1. Systemkontext und Verantwortlichkeiten	21
4.2. Zentrale Entwurfsentscheidungen	23
4.3. Analyse- und Publish-Pfad	24
4.4. Was das Modell ausgeben darf	26
4.5. Sicherheits- und Testbarkeitsziel	27
4.6. Zwischenfazit	27
5. Implementierung	28
5.1. Was eine Person im Issue sieht	28
5.2. Projektaufbau	32
5.3. Vier Stellen, an denen Soll und Ist auseinanderlaufen	33
5.3.1. Zu weite Aktivierung	33
5.3.2. Strukturelle, nicht inhaltliche Modellgrenze	34
5.3.3. Sichtbare Rückfrage, gespeicherter Vollvorschlag	34
5.3.4. Falsche Publish-Reihenfolge	35
5.4. Weitere Grenzen, knapp	36
5.5. Was die Tests tatsächlich tragen	36
6. Qualitätssicherung und Evaluation	37
6.1. Evaluationsfragen	37
6.2. Untersuchungsstand und Artefakte	38
6.3. Prüfprotokoll und Nachverfolgbarkeit	39
6.4. Bewertungslogik	41
6.4.1. Prüfung der Sicherheits- und Freigabegrenzen	41
6.4.2. Trennung technischer und inhaltlicher Prüfung	43
6.5. Befunde entlang der Prozessgrenzen	44
6.5.1. Ereignisannahme und Zugriffsschutz	44
6.5.2. Modellgrenze und Ausgabequalität	44
6.5.3. Speicherung und Publish	45
6.6. Überleitung zu den Ergebnissen	45
7. Ergebnisse	46
7.1. Gesamtstand der Anforderungen	46
7.2. Illustrierter Ablauf	48
7.3. Zustände eines Vorschlags	49
7.4. Reichweite des Ergebnisses	51
8. Diskussion	52
8.1. Interpretation des Ergebnisbeitrags	52
8.1.1. Formale Struktur und fachliche Qualität	52

8.1.2. Rückfragen und Unsicherheit	53
8.2. Nutzenhypothesen, Aufwand und Alternativen	53
8.3. Zielkonflikte der Architektur	55
8.4. Organisatorische Einbettung menschlicher Kontrolle	55
8.5. Gültigkeit der Untersuchung	56
8.6. Risiken und verantwortbarer Einsatz	57
8.7. Übertragbarkeit	58
8.8. Diskussionsfazit	58
9. Fazit und Ausblick	59
9.1. Beantwortung der Forschungsfragen	59
9.1.1. Erforderliche Informationen eines Tickets	59
9.1.2. Kontrollierte Integration eines KI-Assistenten	60
9.2. Wissenschaftlicher und praktischer Beitrag	61
9.3. Priorisierter Ausblick	61
9.4. Schlussfolgerung	62
Quellen	63
A. Illustrierter Ticketverlauf	68
B. Dokumentierter Prüfstand	70
C. Ausgabeschema des Prototyps	71

Abbildungsverzeichnis

4.1. Systemkontext und Komponenten des Dev-Assist-Prototyps mit Vertrauensgrenzen. Der Express-Dienst orchestriert GitLab-Zugriffe, KI-Analyse und lokale Kontextpersistenz. Rote Datenflüsse überschreiten eine Plattform-, Prozess- oder Anbietergrenze. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026. . . .	22
4.2. Sequenz der Analyse- und Publish-Verarbeitung über den GitLab-Webhook. GitLab erhält sofort eine Bestätigung (HTTP 202); die eigentliche Arbeit läuft danach ohne weitere Rückmeldung an den Absender. Rote Hinweise kennzeichnen Fehlerpfade und dass sichtbare Vorschau, gespeicherter Kontext und Freigabe nicht an dieselbe Fassung gebunden sind. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	25
5.1. Anlegen eines unvollständigen Dev-Assist-Issues in GitLab. Die erste inhaltliche Zeile enthält @dev-assist. Der Screenshot stammt aus einem dokumentierten Lauf im untersuchten Projekt.	29
5.2. Klärungskommentar von Dev-Assist und anschließende Antworten im selben Issue. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.	30
5.3. Dasselbe Issue nach @dev-assist publish. Titel und Beschreibung enthalten den gespeicherten Entwurf. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.	31
7.1. Soll-Zustandsmodell eines Dev-Assist-Vorschlags und Abweichung der aktuellen Implementierung. Die Freigabe muss an dieselbe Fassung gebunden sein, die vollständig in GitLab sichtbar war. Der rot gestrichelte Übergang zeigt die fehlende Prüfung von Status, Fassung und Freigabe. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	50

Tabellenverzeichnis

2.1. Einordnung aktueller empirischer Arbeiten im Verhältnis zu Dev-Assist	10
3.1. Informationsgruppen des abgeleiteten Qualitätsmodells	15
3.2. Herkunft und Nachverfolgbarkeit der Anforderungsgruppen	19
4.1. Architekturbereiche, Verantwortlichkeiten und zentrale Module	23
4.2. Fehlerklassen und vorgesehene Reaktionen im Zielablauf	26
5.1. Leistungsstand der Kernfunktionen im untersuchten Prototyp	32
5.2. Projektbereiche und umgesetzte Verantwortlichkeiten	33
6.1. Ergebnisse des dokumentierten Build- und Testlaufs	38
6.2. Untersuchte projektinterne Artefaktgruppen	39
6.3. Prüfprotokoll und zulässige Schlussfolgerungen	40
7.1. Konsolidierter Anforderungs- und Nachweisstand des untersuchten Pro- totyps (eigene Darstellung)	47
8.1. Wirtschaftliche Einordnung der Aufbereitungsalternativen ohne Wir- kungsnachweis	54
8.2. Zentrale Bedrohungen der Gültigkeit und ihre Konsequenzen	56
9.1. Priorisierte Roadmap vom Machbarkeitsnachweis zur belastbaren Eva- luation	62
B.1. Kurzprotokoll des dokumentierten Build- und Testlaufs	70
C.1. Felder des kompakten Ausgabeschemas	71

Abkürzungsverzeichnis

API	Application Programming Interface
CLI	Command Line Interface
FA	funktionale Anforderung
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSONL	JSON Lines
LLM	Large Language Model
NFA	nicht-funktionale Anforderung
NIST	National Institute of Standards and Technology
OCR	Optical Character Recognition
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SRS	Software Requirements Specification
TTL	Time to Live

1. Einleitung

Software-Tickets sind in agilen Teams mehr als Aufgabenlisten. Sie halten fest, was gebaut oder repariert werden soll, und dienen zugleich als Gesprächsraum zwischen meldenden Personen, Product Ownern und Entwicklung. In der Praxis sind diese Tickets häufig unvollständig. Forschung zu Bug Reports zeigt eine wiederkehrende Lücke: Meldende Personen liefern andere Angaben, als Entwicklungsteams für Umsetzung und Prüfung brauchen. Reproduktionsschritte, Testfälle und technischer Kontext fehlen oft oder tauchen erst später in Kommentaren auf (vgl. Bettenburg et al., 2008, S. 308 f.; Breu et al., 2010, S. 301 ff.). Auch kurze User Stories können formal lesbar sein und trotzdem fachlich lückenhaft oder praktisch nicht nutzbar bleiben (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.).

In GitLab liegt dieser Kontext selten an einer Stelle. Titel, Beschreibung und Kommentare müssen vor der Implementierung manuell zusammengeführt werden. Widersprüche sind zu klären, Akzeptanzkriterien oft erst zu formulieren. Ein Assistent, der genau in diesem Arbeitsort mitliest, gezielte Rückfragen stellt und einen strukturierten Entwurf anbietet, könnte diese Aufbereitung unterstützen. Sprachmodelle erzeugen dabei überzeugend formulierte Texte, die unbelegte Aussagen enthalten können. Ein solcher Assistent darf das Ticket deshalb nicht selbst überschreiben. Änderungen brauchen eine sichtbare Prüfung durch einen Menschen.

Diese Arbeit stellt mit Dev-Assist einen solchen Assistenten als technischen Machbarkeitsnachweis vor. Aus der Nutzungssicht reicht ein einfacher Ablauf: Eine Person spricht den Dienst im GitLab-Issue mit `@dev-assist` an, prüft die Antwort im selben Ticket und gibt einen Entwurf gegebenenfalls mit `@dev-assist publish` frei. Wie dieser Ablauf technisch gebaut und begrenzt wird, ist Gegenstand der späteren Kapitel.

Die Problemstellung ist damit doppelt. Fachlich ist unklar, welche Angaben ein Ticket überhaupt enthalten muss, damit Umsetzung und Prüfung nachvollziehbar sind. Technisch ist unklar, wie ein Sprachmodell in genau diesen GitLab-Ablauf eingebunden werden kann, ohne das Ticket unkontrolliert zu verändern. Daraus folgen die beiden Leitfragen der Arbeit:

1. **Forschungsfrage 1:** Welche Informationen muss ein GitLab-Ticket enthalten, damit Entwicklerinnen und Entwickler es nachvollziehbar umsetzen und über-

prüfen können?

2. **Forschungsfrage 2:** Wie lässt sich ein KI-gestützter Assistent so in einen GitLab-basierten Entwicklungsablauf integrieren, dass Analyse, Rückfragen, Vorschlag und Freigabe kontrolliert ablaufen?

1.1. Ziel und Vorgehen

Ziel ist ein technischer Machbarkeitsnachweis, nicht der Ersatz menschlicher Anforderungsarbeit. Die erste Frage klärt, welche Informationsarten ein Ticket entwicklungsfähig machen. Die zweite Frage klärt, welche technischen Grenzen nötig sind, wenn ein Sprachmodell in GitLab eingebunden wird. Beide hängen zusammen: Das fachliche Qualitätsmodell bestimmt, welche Felder ein Vorschlag enthalten soll. Die technischen Grenzen bestimmen, was das Modell allein entscheiden darf.

Die Arbeit geht deshalb vom Problem zum Artefakt. Aus der Literatur entsteht ein Qualitätsmodell, daraus Anforderungen, Architektur und ein Prototyp. Die Evaluation prüft diesen Prototyp gegen das Soll. Die Literatúrauswahl ist keine vollständige systematische Studie. Sie konzentriert sich auf Qualitätsmerkmale von User Stories und Bug Reports, auf automatisierte Qualitätshilfen und auf Risiken generativer Systeme. Betrachtet wird jeweils ein Analyse- oder Freigabevorgang, den ein GitLab-Ereignis auslöst. Ticketqualität meint hier, ob ein Ticket umsetzbar und prüfbar ist. Sie meint nicht die Qualität des fertigen Produkts und nicht die Leistung des Teams.

Literatur begründet das fachliche Soll, die GitLab-Dokumentation die Schnittstellen und der Programmstand das technische Ist. Build und Tests belegen Eigenschaften der festen Programmteile, also der Teile ohne Sprachmodell. Daraus folgt weder eine fachlich richtige Modellantwort noch ein sicherer Produktivbetrieb. Eine Feldstudie oder ein Wirksamkeitsnachweis realer Modellantworten ist nicht Bestandteil der Arbeit.

1.2. Aufbau und Abgrenzung

Kapitel 2 legt die fachlichen und technischen Grundlagen dar. Kapitel 3 überführt sie in Anforderungen, Kapitel 4 in die Architektur und Kapitel 5 in die Implementierung. Kapi-

tel 6 beschreibt die Evaluation. Darauf aufbauend stellen Kapitel 7 und 8 Ergebnisse und Diskussion getrennt dar. Kapitel 9 beantwortet die Forschungsfragen und priorisiert weiteren Bedarf. Ein illustrierter Ticketverlauf, der dokumentierte Prüfstand und das Ausgabeschema stehen im Anhang.

Nutzerexperimente, Langzeitdaten und ein statistisch ausgewerteter Ticketkorpus gehören nicht zur Untersuchung. Anhänge und Screenshots werden nicht inhaltlich analysiert. Die Arbeit kann deshalb technische Machbarkeit und konkrete Gestaltungsgrenzen zeigen. Eine allgemeine Zeitersparnis oder Qualitätssteigerung im Feld lässt sich damit nicht nachweisen. Diese Grenze gilt für die Interpretation der Evaluation und der Forschungsfragen.

2. Theoretische und technische Grundlagen

Dieses Kapitel bleibt bei den Grundlagen. Es klärt, woran Ticketqualität erkennbar ist, was Sprachmodelle leisten und wo ihre Grenzen liegen, welche empirischen Arbeiten dazu vorliegen, wie GitLab als Arbeitsort funktioniert und welche Risiken daraus folgen. Kurze Schlussfolgerungen für die spätere Gestaltung stehen am Ende der Abschnitte. Sie beschreiben noch kein fertiges System. Die technische Umsetzung folgt in den Kapiteln 4 und 5.

2.1. Requirements Engineering und Ticketqualität

Requirements Engineering umfasst das Erheben, Analysieren, Dokumentieren, Prüfen und Verwalten von Anforderungen über ihren Lebenszyklus. Die öffentlich zugängliche Katalogseite zu ISO/IEC/IEEE 29148 beschreibt diesen Prozessbereich und die zugehörigen Informationsprodukte (vgl. ISO/IEC/IEEE, 2018, o. S.). Der Volltext der Norm wird hier nicht vorausgesetzt. In der Fachliteratur werden mit diesem Standard Merkmale wie Eindeutigkeit, Vollständigkeit, Nachverfolgbarkeit und Prüfbarkeit verbunden und in beobachtbare Kriterien übersetzt (vgl. Lubos et al., 2024, S. 389–397; Paiva et al., 2026, Abschnitte 3 und 5).

Ein GitLab-Ticket ist keine vollständige Spezifikation. Es ist ein kurzes, fortgeschriebenes Arbeitsdokument. Die genannten Merkmale helfen trotzdem, Aussagen und Lücken zu beurteilen: Ist der Gegenstand eindeutig? Fehlen prüfbare Kriterien? Sind Belege und Annahmen unterscheidbar? Welche Felder ein Ticket in dieser Arbeit braucht, wird aus den folgenden empirischen Befunden abgeleitet, nicht aus einer Prüfung gegen den Normvolltext.

2.1.1. Empirische Befunde zu User Stories und Bug Reports

In agilen Prozessen entstehen Anforderungen häufig als User Stories oder Tickets. Das erleichtert iterative Zusammenarbeit und erhöht zugleich die Abhängigkeit vom gemeinsamen Kontext. Inayat et al. beschreiben agiles Requirements Engineering deshalb als stark kommunikations- und abstimmungsorientiert (vgl. Inayat et al., 2015, S. 915 ff.).

Lucassen et al. unterscheiden bei User Stories drei Ebenen: die Form eines Textes, seine Bedeutung und seine praktische Nutzbarkeit (vgl. Lucassen et al., 2016, S. 386 f.). Ein Text kann lesbar sein und trotzdem die falsche Bedeutung vermitteln oder für die Umsetzung unbrauchbar bleiben. Diese Dreiteilung bleibt später wichtig: Eine sauber gegliederte Ausgabe ist noch keine gute Anforderung.

Bug Reports konkretisieren, welche Informationen in technischen Tickets fehlen können. Bettenburg et al. zeigen, dass Entwicklungsteams Reproduktionsschritte, Stack Traces und Testfälle besonders hilfreich finden, meldende Personen diese Angaben aber nicht immer bereitstellen können (vgl. Bettenburg et al., 2008, S. 308 f.). Breu et al. untersuchten Fragen in 600 Bug Reports und identifizierten Nachfragen zu Reproduktion, Version, Betriebssystem, Beispielen, Ausgaben und Screenshots (vgl. Breu et al., 2010, S. 301 und S. 303). Chaparro et al. heben beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte als zentrale, zugleich häufig fehlende Bestandteile hervor (vgl. Chaparro et al., 2017, S. 396 f.).

Diese Befunde gelten nicht unverändert für jede Ticketart. Sie begründen aber einen gemeinsamen Kern. Ein Ticket braucht einen eindeutigen Gegenstand, Ziel und Kontext, erwartetes oder gewünschtes Verhalten, eine Abgrenzung, überprüfbare Kriterien und sichtbare offene Punkte. Bei Fehlern kommen Ist-Verhalten, Soll-Verhalten und Reproduktion hinzu. Bei neuen Funktionen stehen Nutzen, Umfang und Akzeptanzbedingungen im Vordergrund. Technische Evidenz darf eine fehlende fachliche Entscheidung nicht durch Annahmen ersetzen.

GitLab-Issues sind keine statischen Texte. Die Beschreibung ist der Ausgangspunkt. Rückfragen und Entscheidungen entstehen in Kommentaren. Ein Assistenzsystem muss den Gesprächsverlauf mitlesen und geklärte Informationen ins Ticket zurückführen. Qualität bedeutet hier nicht maximalen Umfang, sondern die Trennung von Aussage, Beleg und Unsicherheit.

2.1.2. Ableitung des Qualitätsmodells

Aus diesen Befunden folgt ein ticketartübergreifendes, aber nicht starres Modell. **Ziel und Gegenstand** identifizieren Änderung und Nutzen. **Kontext und Scope** erfassen Voraussetzungen, Abhängigkeiten und Nicht-Ziele. **Verhalten und Kriterien** verbinden erwartete Wirkung mit prüfbaren Bedingungen; bei Fehlern ergänzen sie Ist-Verhalten

und Reproduktion. **Evidenz und Hinweise** trennen belegte technische Angaben von fachlichen Entscheidungen. **Unsicherheit** hält Fragen, Widersprüche, Risiken und Annahmen sichtbar.

Die Indikatoren sind beobachtbar und in der zitierten Literatur verankert. Eindeutigkeit zeigt sich an einem klar bezeichneten Gegenstand und an kenntlich gemachten Widersprüchen (vgl. Lucassen et al., 2016, S. 386 f.; Lubos et al., 2024, S. 389–397). Prüfbarkeit zeigt sich an Akzeptanzkriterien, die ein Ergebnis statt einer Implementierungsaktivität beschreiben (vgl. Paiva et al., 2026, Abschnitte 3 und 5). Nachverfolgbarkeit verlangt, belegte Ticket-, Kommentar- und Repository-Angaben von Aussagen zu trennen, die das Modell selbst ergänzt hat (vgl. Tizard et al., 2023, S. 2381 f.). Vollständigkeit ist ticketartspezifisch: Reproduktionsschritte bei Fehlern, Nutzen und Abgrenzung bei Funktionen (vgl. Bettenburg et al., 2008, S. 308 f.; Chaparro et al., 2017, S. 396 f.). Ein ausgefülltes Feld beweist für sich noch kein Qualitätsmerkmal.

Für die spätere Gestaltung folgt daraus: Zuerst ist zu prüfen, welche Kategorien für die Ticketart relevant und belegt sind. Wesentliche Lücken sollen zu konkreten Rückfragen führen. Bei belastbarem Kern kann ein Vorschlag entstehen, in dem Unsicherheit sichtbar bleibt. Ein kurzer, belegter Entwurf kann geeigneter sein als ein ausführlicher Text mit Annahmen. Kapitel 3 übersetzt diese Kategorien in Anforderungen.

2.2. Large Language Models und agentenbasierte Assistenz

Große Sprachmodelle (Large Language Models) werden auf großen Textmengen vortrainiert und können Aufgaben anhand natürlichsprachlicher Anweisungen und Beispiele bearbeiten. Brown et al. zeigen mit GPT-3, dass Aufgaben ohne neues Training gelöst werden können, wenn die Aufgabe im Eingabetext beschrieben wird (vgl. Brown et al., 2020, S. 1877 und S. 1879). Die Gestaltung dieses Eingabetextes wird als Prompt Engineering bezeichnet (vgl. Zhao et al., 2023, S. 45 f.). Für Ticketarbeit ist das relevant, weil Ticketart, Sprache und Informationslage stark variieren und sich nicht vollständig in festen Einzelregeln abbilden lassen.

Ein Modell kann vorhandene Aussagen gruppieren, fehlende Kategorien markieren, Rückfragen formulieren und einen strukturierten Entwurf erzeugen. Das ist nicht mit gesichertem Systemverständnis gleichzusetzen. Das Modell kennt weder automatisch

die fachliche Absicht noch den vollständigen Projektkontext. Seine Rolle bleibt Assistenz: Es liefert einen Entwurf. Die umgebende Anwendung muss prüfen, was davon übernommen wird.

Solche Modelle können außerdem Werkzeuge aufrufen und Daten aus der Umgebung einbeziehen (vgl. Xi et al., 2023, S. 1; Yao et al., 2023, S. 1 f.; Schick et al., 2023, S. 68539 f.). Entscheidend ist weniger das einzelne Verfahren als das Prinzip: Ein Modell wird erst durch kontrollierte Datenquellen, Schnittstellen und Berechtigungen zum Bestandteil eines Entwicklungsablaufs. Für die Grundlagen reicht: Das Modell darf GitLab nicht selbst beschreiben. Lesen, Speichern und Freigabe bleiben Aufgaben der Anwendung. Die konkrete Einbindung folgt in den Kapiteln 4 und 5.

2.3. Automatisierte Unterstützung und aktuelle LLM-Forschung

2.3.1. Regelbasierte und hybride Werkzeuge

Ticketqualität lässt sich teilweise automatisiert unterstützen. CUEZILLA bewertet Bug Reports und gibt Hinweise auf ergänzende Informationen (vgl. Bettenburg et al., 2008, S. 308 f.). AQUASA macht formale und sprachliche Defekte in User Stories sichtbar (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). DeMIBuD erkennt fehlendes erwartetes Verhalten und fehlende Reproduktionsschritte (vgl. Chaparro et al., 2017, S. 397 und S. 401 ff.). BEE klassifiziert Bug Reports, markiert Observed Behavior, Expected Behavior und Steps to Reproduce und meldet fehlende Elemente (vgl. Song/Chaparro, 2020, S. 1551 f.). ChatBR kombiniert einen Klassifikator mit einem Sprachmodell, das als fehlend erkannte Bestandteile ergänzt. Über sechs Projekte berichten Bo et al. eine hohe strukturelle Vollständigkeit und eine mittlere semantische Ähnlichkeit der Ergänzungen von 84,10 Prozent (vgl. Bo et al., 2024, S. 1472–1483). Strukturelle Anschlussfähigkeit ist damit nicht dasselbe wie faktische Richtigkeit im konkreten Projekt.

Diese Werkzeuge werden hier nicht als Bausteine übernommen. Sie sind Forschungsprototypen für jeweils eine Artefaktart und eine Prüfaufgabe. Sie liegen nicht als GitLab-Dienste vor, die auf neue Kommentare reagieren, und erzeugen keine Ticketbeschreibung, die ein Mensch freigeben muss. Übernommen werden die Qualitätskategorien und die Trennung von Lücken erkennen, Text erzeugen und Freigabe. Die Werkzeuge selbst wären eine mögliche spätere Prüfstufe, nicht der Gegenstand dieser Arbeit.

2.3.2. Empirische Arbeiten zu LLMs im Requirements Engineering

Die jüngere Forschung untersucht Sprachmodelle für konkrete RE-Aufgaben: Generierung oder Überarbeitung, Qualitätsprüfung und Verknüpfung verteilter Quellen. Die Studien liefern erste Evidenz. Formal überzeugende Texte und ein kontrollierter Arbeitsprozess bleiben getrennte Gegenstände.

Krishna et al. vergleichen GPT-4 und CodeLlama bei Erstellung, Prüfung und Korrektur vollständiger Anforderungsspezifikationen. Potenzial für strukturierte Entwürfe und Zeitersparnis zeigte sich, fiel aber nach Modell und Aufgabe unterschiedlich aus (vgl. Krishna et al., 2024, S. 475–483). Die Studie arbeitet mit langen Spezifikationsdokumenten, nicht mit kurzen GitLab-Tickets, die durch Kommentare ergänzt werden. Ein Nachweis, dass ein Assistent in GitLab Zeit spart, folgt daraus nicht.

Lubos et al. setzen ein LLM als Prüfinstanz für Qualitätsmerkmale nach ISO/IEC/IEEE 29148 ein. Die Übereinstimmung zwischen Menschen und Modell war teilweise schwach. Erklärungen und überarbeitete Formulierungen wurden dennoch häufig als plausibel eingeschätzt (vgl. Lubos et al., 2024, S. 389–397). Das stützt einen assistierenden Einsatz und warnt davor, ein Modellurteil mit fachlichem Konsens gleichzusetzen.

Paiva et al. erzeugen aus Issue-Titeln von fünf Open-Source-Projekten Anforderungssätze und bewerten, ob sie eindeutig, prüfbar und auf jeweils einen Sachverhalt begrenzt sind. Die Autoren beschränken ihre Aussage ausdrücklich auf sprachlich wohlgeformte Sätze bei sehr wenig Eingangskontext. Ob die Sätze zum konkreten Projekt passen und mit den Beteiligten abgestimmt sind, wird nicht nachgewiesen (vgl. Paiva et al., 2026, Abschnitte 3 und 5). Für die spätere Gestaltung spricht das dafür, Beschreibung, Kommentare und begrenzten Repository-Kontext gemeinsam zu betrachten. Ob dieser zusätzliche Kontext tatsächlich bessere Vorschläge erzeugt, prüft diese Arbeit nicht empirisch.

K C et al. erzeugen zu 476 Feature Requests konkrete Rückfragen. 164 von 167 manuell bewerteten Fragen galten als korrekt. Erfahrene Entwicklerinnen und Entwickler bestätigten, dass Nachfragen bei mangelhaften Requests typischerweise der erste Schritt sind (vgl. K C et al., 2025, S. 104–116). Semantische Mehrdeutigkeiten blieben schwieriger als lokale Sprachdefekte. Der Befund stützt den dialogischen Ansatz, nicht die automatische Freigabe.

Weitere industrielle Studien bestätigen dieselbe Grenze mit unterschiedlichen Aufgaben. Projektnahe Beispiele verbessern die Erkennung von Mehrdeutigkeit, bleiben aber von Datensatz und Domäne abhängig (vgl. Bashir et al., 2025, S. 620–631). Präzisionen für Unvollständigkeit, Inkonsistenz und Ambiguität fallen in einer medizinischen Spezifikation deutlich unter eine belastbare Automatisierungsschwelle (vgl. Mahbub et al., 2024, S. 171972–171992). Zusätzlicher Dokumentkontext hilft, garantiert aber keine korrekte Antwort (vgl. Ibtasham et al., 2025, S. 277–292; Tizard et al., 2023, S. 2381 f. und S. 2389 ff.). Frattini et al. trennen deshalb die Eigenschaft einer Anforderung, ihre Bewertung und die Wirkung im Nutzungskontext (vgl. Frattini et al., 2023, S. 507–520). Ein sprachlich klarer Vorschlag ist zuerst ein formales Artefakt. Ob er Implementierung oder Abstimmung verbessert, braucht eigene Nutzungsdaten.

Tabelle 2.1: Einordnung aktueller empirischer Arbeiten im Verhältnis zu Dev-Assist

Arbeit	Untersuchter Beitrag	Konsequenz für diese Arbeit
Krishna et al. (2024)	Generierung und Prüfung vollständiger Anforderungsspezifikationen mit zwei Sprachmodellen	Potenzial ist aufgaben- und modellabhängig. Die Ticketwirkung ist separat zu prüfen
Lubos et al. (2024)	LLM-basierte Qualitätsprüfung nach ISO/IEC/IEEE 29148 mit menschlicher Bewertung	Die Modellerklärung unterstützt die Prüfung, ersetzt aber keinen fachlichen Konsens
K C et al. (2025)	Erkennung von Lücken und Generierung konkreter Rückfragen für Feature Requests	Dialog unterstützt die Verfeinerung. Semantische Lücken bleiben menschlich zu prüfen
Bo et al. (2024)	Hybride Erkennung und generative Ergänzung fehlender Bug-Report-Bestandteile	Strukturelle Vollständigkeit ist nicht mit faktischer Richtigkeit gleichzusetzen
Tizard et al. (2023)	Verknüpfung von Forum, Issue Tracker und Dokumentation	Kontextquellen gemeinsam betrachten und die Herkunft erhalten
Paiva et al. (2026)	Anforderungsgenerierung aus Issue-Titeln mit mehreren Prompts und Modellen	Sprachqualität bei Minimalkontext belegt keine Projektangemessenheit
Ibtasham et al. (2025)	Industrielle kontextgestützte Beantwortung von Requirements-Fragen	Der Abruf zusätzlichen Kontexts unterstützt, garantiert aber keine korrekte Antwort
Frattini et al. (2023)	Theorie und Forschungsrahmen für Requirements Quality	Formale Artefaktqualität von der Wirkung im Entwicklungsprozess trennen

Tabelle 2.1 macht den Unterschied der Untersuchungsgegenstände sichtbar. Die zitierten Arbeiten prüfen Generierung, Qualitätsurteil, Rückfragen oder Kontextabruf jeweils für sich. Keine von ihnen untersucht denselben GitLab-Ablauf aus Ereignis, Rückfrage, sichtbarer Vorschau und Freigabe. Genau diesen Ablauf soll der spätere Prototyp bauen. Ob die entstehenden Texte fachlich brauchbar sind, ist damit nicht gezeigt und wird in dieser Arbeit auch nicht empirisch gemessen.

2.4. GitLab als Arbeits- und Integrationsplattform

2.4.1. Entwicklungsplattform

GitLab ist eine webbasierte Plattform für den Softwarelebenszyklus. Teams verwalten dort Quellcode, Merge Requests, CI/CD und Aufgaben in einem gemeinsamen Projekt. Der Issue Tracker ist dabei der Ort, an dem Anforderungen, Fehler und technische Aufgaben dokumentiert und diskutiert werden. Für diese Arbeit zählt GitLab deshalb nicht als beliebige Schnittstelle, sondern als der Arbeitsort, in dem Ticketqualität sichtbar wird und in dem ein Assistent ohne zusätzliche Oberfläche nützlich sein muss.

2.4.2. Issues und Kommentarverlauf

Ein Issue besteht aus Titel, Beschreibung, Metadaten und einem Kommentarverlauf. Die Erstbeschreibung ist nur der Ausgangspunkt. Rückfragen, Antworten und Entscheidungen entstehen später als Kommentare (in GitLab Notes). Ein Assistent, der nur den Anfangstext sieht, verfehlt den tatsächlichen Informationsstand. Über die GitLab-Schnittstellen lassen sich Titel und Beschreibung lesen und ändern sowie Kommentare lesen und ergänzen (vgl. GitLab Documentation, o. J.c, o. S.; GitLab Documentation, o. J.d, o. S.). Genau diese beiden Schreibwege bilden später die sichtbare Rückfrage und die Freigabe des Entwurfs.

2.4.3. Webhooks als Integrationspunkt

Externe Dienste werden über Webhooks angebunden. Ein Webhook ist eine automatische Benachrichtigung: Bei einem konfigurierten Ereignis sendet GitLab eine HTTP-Anfrage mit den Ereignisdaten. Für den späteren Ablauf sind neue oder geänderte Issues sowie neue Kommentare die relevanten Auslöser (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook events"; GitLab Documentation, o. J.b, Abschnitt "Comment events"). Der Empfänger soll schnell antworten und längere Arbeit danach erledigen (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook receiver requirements"). Ob eine Lieferung wirklich von GitLab stammt, ob sie nicht wiederholt eingeschleust wird und ob Bot-Kommentare von Nutzerkommentaren getrennt bleiben, sind damit Bedingungen einer kontrollierten Einbindung. Das konkrete Prüfverfahren gehört in die Kapitel 4

und 5.

2.5. Risiken generativer Assistenz

Die folgenden Risiken stammen aus der Fachliteratur. Sie begründen, warum ein Ticket-Assistent nicht als freier Textgenerator entworfen werden darf. Die konkrete Absicherung ist Gegenstand der Architektur, nicht dieses Kapitels.

Das zentrale inhaltliche Risiko sind falsche oder unbelegte Modellinhalte. Huang et al. beschreiben Halluzinationen als plausibel wirkende, aber nicht faktische Aussagen (vgl. Huang et al., 2023, S. 1 f.). NIST verwendet dafür auch den Begriff Confabulation (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Eine Prüfung der Datenform kann falsche Feldtypen abweisen. Ob ein syntaktisch gültiger Satz wahr ist, kann sie nicht feststellen. Unsicherheit muss deshalb sichtbar bleiben. Bei wesentlichen Lücken sind Rückfragen zu stellen. Vor dem Überschreiben des Issues ist eine menschliche Prüfung zu verlangen.

Ein zweites Risiko ist Prompt Injection: Tickettexte oder eingebundene Repository-Inhalte können versteckte Anweisungen enthalten, die das Modell umleiten sollen (vgl. Liu et al., 2023, S. 1; OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection"). Diese Inhalte sind als Daten zu behandeln, nicht als Befehle. Das Modell darf deshalb keine freie Werkzeugauswahl und keine produktive Schreibberechtigung erhalten. OWASP bezeichnet zu weitreichende Rechte eines Modells als übermäßige Handlungsfähigkeit (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency").

Ein drittes Risiko entsteht an der Webhook-Grenze selbst: gefälschte Ereignisse, wiederholte Lieferungen und Bot-Schleifen. Diese Fälle müssen außerhalb des Modells behandelt werden. Aus der Literatur folgen vier Leitprinzipien. Erstens wird nur explizit adressierter Kontext verarbeitet. Zweitens sammelt die Anwendung Daten kontrolliert und trennt sie von festen Systemanweisungen. Drittens akzeptiert sie nur eine definierte, maschinenlesbare Antwort. Viertens erfolgen produktive Änderungen nach einer sichtbaren Vorschau und einem gesonderten Freigabekommando. Kapitel 3 übersetzt diese Prinzipien in Anforderungen.

2.6. Zwischenfazit

Für die erste Forschungsfrage zeigen Requirements Engineering sowie User-Story- und Bug-Report-Forschung, dass Ticketqualität mehrere Informationsarten umfasst. Ziel, Kontext, erwartetes Verhalten, Prüfkriterien und Unsicherheit bilden einen gemeinsamen Kern. Ticketartspezifische Evidenz ergänzt ihn. Kommentare gehören zum Artefakt, weil fehlende Informationen häufig erst im Dialog entstehen. Strukturelle, semantische und pragmatische Qualität sind getrennt zu beurteilen.

Für die zweite Forschungsfrage zeigen Issues, Notes und Webhooks, wie ein Modell technisch in GitLab eingebunden werden kann. Halluzination, Prompt Injection und übermäßige Handlungsfähigkeit machen zugleich deutlich, dass Modellflexibilität keine hinreichende Architektur ist. Kontrolle entsteht außerhalb des Modells: durch Authentifizierung, Kontextbegrenzung, minimale Berechtigungen, Laufzeitvalidierung und explizite Freigabe. Ob zusätzliche Kontextquellen die fachliche Qualität der Vorschläge verbessern, bleibt empirisch offen. Kapitel 3 übersetzt diese Annahmen in Anforderungen.

3. Anforderungsanalyse

Dieses Kapitel übersetzt die Grundlagen in ein Soll für Dev-Assist. Es beginnt beim heutigen GitLab-Alltag, beschreibt den gewünschten Zielablauf und die beteiligten Rollen und leitet daraus das Qualitätsmodell ab. Erst danach folgen die nummerierten funktionalen und nicht-funktionalen Anforderungen. Die Herkunftstabelle kommt zum Schluss, wenn die Nummern bekannt sind.

3.1. Ausgangs- und Zielprozess

Im Ausgangsprozess erstellt eine Person ein Issue mit einer Anforderung, einem Fehler oder einer technischen Aufgabe. Fehlende Informationen werden in Kommentaren nachgefragt. Antworten, Screenshots und Entscheidungen ergänzen den Kontext schrittweise. Vor der Umsetzung muss eine bearbeitende Person diese Fragmente zusammenführen. Die Arbeit geht von der Annahme aus, dass diese Zersplitterung Wartezeiten, wiederholte Rückfragen und schwer nachvollziehbare Übergaben begünstigt. Eine Prozessmessung dazu ist nicht Bestandteil der Untersuchung.

Dev-Assist soll diese Zusammenarbeit unterstützen, ohne sie zu ersetzen. Der Zielprozess umfasst fünf Schritte. Eine Person adressiert den Assistenten zu Beginn der ersten inhaltlichen Zeile mit `@dev-assist`. Die Anwendung prüft das Ereignis und sammelt Issue- sowie Kommentar-Kontext. Bei wesentlichen Lücken veröffentlicht sie gezielte Rückfragen. Andernfalls erstellt sie einen strukturierten Vorschlag. Der Vorschlag bleibt zunächst sichtbar und unverbindlich. Erst `@dev-assist publish` übernimmt den gespeicherten Stand in das Issue.

3.2. Stakeholder und Priorisierung

Die Interessen der Beteiligten stehen teilweise in Spannung. Meldende Personen brauchen eine geringe Einstiegshürde. Entwicklung und Qualitätssicherung brauchen genug Kontext für Annahmen und Prüfkriterien. Product Owner betrachten Nutzen und Umfang, Betreiber sichere Zugriffe und Fehlerdiagnose. Der Assistent darf diesen Ausgleich nicht durch erfundene Inhalte herstellen. Er muss entscheidungsrelevante Lü-

cken gezielt sichtbar machen.

Muss-Anforderungen sichern den Kernprozess und die Sicherheitsgrenzen: Aktivierung, Abruf des Ticketkontexts, strukturierte Analyse, Speicherung, expliziter Publish sowie die Begrenzung externer Eingänge und Modellantworten. Soll-Anforderungen wie erweiterte Screenshot-Verarbeitung, zusätzliche GitLab-Metadaten und ausführlichere Diagnosen erhöhen den Nutzen. Dashboard, Produktmetriken und ein zusätzliches Rollenmodell bleiben Kann-Erweiterungen. Diese Priorisierung beschreibt das Soll. Sie behauptet nicht, dass der Repository-Stand sämtliche Muss-Anforderungen erfüllt. Innerhalb des Muss-Umfangs haben Integrität und menschliche Kontrolle Vorrang vor Komfort.

3.3. Qualitätsmodell für Ticketvorschläge

Ein Ticket gilt in dieser Arbeit als entwicklungsfähig, wenn sein Gegenstand verstanden, sein Umfang abgegrenzt, die Umsetzung gegen beobachtbare Kriterien geprüft und verbleibende Unsicherheit erkannt werden kann. Tabelle 3.1 fasst die daraus abgeleiteten fünf Informationsgruppen zusammen.

Tabelle 3.1: Informationsgruppen des abgeleiteten Qualitätsmodells

Informationsgruppe	Inhalt und Zweck
Ziel und Gegenstand	Eindeutiger Titel, Problem oder gewünschter Nutzen sowie betroffene Rollen oder Systeme
Kontext und Scope	Relevante Voraussetzungen, Randbedingungen, Abhängigkeiten und bewusste Nicht-Ziele
Verhalten und Kriterien	Erwartetes Ergebnis, bei Fehlern Ist-/Soll-Verhalten und Reproduktion, außerdem prüfbare Akzeptanzkriterien
Evidenz und Hinweise	Belegte Logs, Fehlermeldungen, Plattform- und Repository-Informationen ohne erfundene Implementierungsdetails
Unsicherheit	Offene Fragen, Widersprüche, Risiken, Annahmen und erforderliche Validierungsschritte

Das fachliche Zielmodell sieht neben Titel und Beschreibung auch Quellenbasis, Um-

fang, Akzeptanzkriterien, technische Hinweise, offene Fragen, Risiken und Prüfschritte vor. Es ist absichtlich breiter als ein Minimalformat, weil eine formal vollständige Ausgabe fehlende Informationen nicht verdecken darf. Kann ein Bereich nicht belastbar gefüllt werden, muss er als offen gekennzeichnet werden. Kapitel 4 und 5 beschreiben ein kompakteres Ausgabeschema mit sechs Feldern. Ob diese Verkürzung reicht, bewertet erst die Evaluation.

Für die Rollen ergeben sich unterschiedliche Nutzungskriterien. Rückfragen an meldende Personen sollen fachliches Ziel, beobachtbares Verhalten und Erfolgskriterien betreffen, nicht interne Dateipfade. Product Owner benötigen eine klare Abgrenzung und sichtbare Annahmen. Entwicklung und Qualitätssicherung brauchen einen Entwurf, aus dem Umsetzung und Prüfung getrennt hervorgehen. Für Betreiber sind Authentifizierung, Konfiguration, Logging und ein sicherer Fehlerzustand entscheidend.

3.4. Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben den kontrollierten Kernablauf. Ihre Nummerierung wird in der Evaluation wieder aufgegriffen.

FA-1 Explizite Aktivierung. Dev-Assist reagiert nur, wenn die erste inhaltliche Zeile einer Issue-Beschreibung oder eines Kommentars mit `@dev-assist` beginnt. Führende Leerzeichen und einfache Markdown-Präfixe dürfen toleriert werden.

FA-2 Relevante Ereignisse. Issue- und Kommentar-Ereignisse müssen in ein gemeinsames internes Format mit Projekt, Issue, Text, Autor und Aktion überführt werden.

FA-3 Kommandotrennung. Eine normale Aktivierung startet die Analyse; `@dev-assist publish` startet ausschließlich die Übernahme eines zuvor gespeicherten Vorschlags.

FA-4 Abruf des Ticketkontexts. Soweit verfügbar, sind das aktuelle Issue, der Kommentarverlauf und optional begrenzter Repository-Kontext einzubeziehen. Webhook-Daten dienen als Fallback.

FA-5 Gemeinsame Analyse. Titel, Beschreibung, Kommentare, erkannte Antworten und Zusatzkontext sind gemeinsam auszuwerten. Gesicherte Aussagen, technische Hinweise und offene Punkte müssen unterscheidbar bleiben.

FA-6 Gezielte Rückfragen. Fehlen wesentliche fachliche Angaben, stellt das System konkrete, einzeln beantwortbare Fragen zu Ziel, Scope, erwartetem Verhalten oder Prüfkriterien. Bereits beantwortete Fragen sollen nicht wiederholt werden.

FA-7 Strukturierter Vorschlag. Bei ausreichendem Kontext erzeugt Dev-Assist einen als Issue-Beschreibung nutzbaren Entwurf. Unbelegte Inhalte dürfen nicht als Tatsachen dargestellt werden.

FA-8 Maschinenlesbares Ausgabeschema. Die Modellantwort muss einem definierten JSON-Schema entsprechen und vor jeder weiteren Verarbeitung geparkt und strukturell validiert werden.

FA-9 Sichtbare Rückgabe. Rückfragen, Vorschläge und Analysefehler werden als gekennzeichnete GitLab-Kommentare ausgegeben.

FA-10 Persistenz. Der analysierte Stand wird unter einem reproduzierbaren, gegen Pfadmanipulation begrenzten Speicherpfad als Markdown und optionale Metadaten gespeichert.

FA-11 Kontrollierter Publish. Publish liest ausschließlich den gespeicherten Stand, aktualisiert zuerst Titel und Beschreibung und bereinigt erst nach erfolgreicher Änderung Dev-Assist-Kommentare. Ohne erfolgreiche Analyse findet keine Übernahme statt.

3.5. Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen begrenzen Betrieb und Qualität. **NFA-1** fordert getrennte Module für Transport, GitLab, Analyse, Formatierung, Persistenz und Publish. **NFA-2** verlangt lokale Testbarkeit ohne reales GitLab und ohne kostenpflichtigen Modellaufruf. **NFA-3** betrifft Fallbacks und einen sicheren Abbruch bei ungültigem Kontext oder ungültiger Modellantwort. **NFA-4** fordert nachvollziehbare Konsolenlogs ohne Secrets.

NFA-5 begrenzt KI-Risiken durch feste Anweisungen, Schema-Validierung, sichtbare Unsicherheit und menschliche Freigabe. **NFA-6** fordert, dass Webhooks standardmäßig als echt erkannt werden müssen und alte oder wiederholte Lieferungen nicht als neu

gelten; unsicherer Betrieb darf nur ausdrücklich für lokale Entwicklung aktiviert werden. **NFA-7** verhindert, dass der Assistent auf eigene Kommentare reagiert und dieselbe Lieferung kurz hintereinander doppelt verarbeitet. **NFA-8** trennt eine schnelle Empfangsbestätigung an GitLab von der längeren Modellarbeit. **NFA-9** verlangt Konfiguration über Umgebungsvariablen und die Trennung von Secrets und Quellcode. **NFA-10** erhält die Bedienung vollständig im GitLab-Workflow und verlangt keine zusätzliche Oberfläche.

Die Anforderungen werden für den Machbarkeitsnachweis als Muss-Anforderungen behandelt, soweit sie den Kernprozess und seine Sicherheitsgrenzen betreffen. Erweiterte Bildanalyse, zusätzliche GitLab-Metadaten und differenzierte Fehlerdiagnosen sind Soll-Anforderungen. Dashboard, quantitative Produktmetriken und ein erweitertes Rollenmodell sind mögliche spätere Funktionen.

3.6. Herkunft und Nachverfolgbarkeit

Die Anforderungen stammen aus vier Quellen. Erstens liefern die in Kapitel 2 ausgewerteten Arbeiten die Qualitätskategorien. Zweitens zeigt der manuelle Ist-Prozess, an welchen Stellen Kontext verteilt ist. Drittens beschreibt die projektinterne Zielsetzung Aktivierung, Analyse, Klärung und Publish. Viertens begrenzen GitLab-Dokumentation, Sicherheitsrisiken und der Technologie-Stack die technische Ausgestaltung.

Tabelle 3.2 ordnet diese Quellen den nun eingeführten Anforderungen zu. Die Zuordnung verhindert, dass das Soll nur aus dem vorhandenen Code rückwärts abgeleitet wird. Der Prototyp wird später gegen dieses Zielbild geprüft. Abweichungen bleiben sichtbar.

Tabelle 3.2: Herkunft und Nachverfolgbarkeit der Anforderungsgruppen

Quelle	Abgeleiteter Bedarf	Anforderungsbezug
Forschung zur Ticketqualität	Fehlende Angaben erkennen, Verhalten und Prüfbarkeit trennen	Qualitätsmodell, FA-5 bis FA-8
GitLab-Ist-Prozess	Beschreibung und Kommentare gemeinsam behandeln, Interaktion im Issue halten	FA-1 bis FA-4, FA-6, FA-9
KI- und Sicherheitsrisiken	Modellantwort begrenzen, Unsicherheit erhalten, menschlich freigeben	FA-8, FA-11, NFA-3, NFA-5 bis NFA-7
Betrieb und Evaluation	Module isolieren, Ereignisse nachvollziehen und externen Zugriff konfigurieren	NFA-1, NFA-2, NFA-4, NFA-8 bis NFA-10

Die Nummerierung stellt Nachverfolgbarkeit her. Kapitel 4 ordnet den Anforderungen Architekturgrenzen zu. Kapitel 5 beschreibt, was der Prototyp tatsächlich tut. Kapitel 6 prüft beides gegen das Soll. Ein späterer Test belegt nur die konkret geprüfte Eigenschaft. Er darf hier noch nicht als Nachweis gelten, dass eine Anforderung erfüllt oder verletzt ist.

3.7. Abgrenzung und Zwischenfazit

Dev-Assist implementiert keinen Produktivcode, erstellt keine Merge Requests und priorisiert keine Tickets. Er ersetzt weder fachliche Entscheidungen noch die Prüfung eines Vorschlags. Die Arbeit bewertet die Struktur und technische Verarbeitung von Ticketinformationen, nicht die Qualität des gesamten Projekts. Eine eigene Bilderkennung, die Text aus Screenshots ausliest, ist nicht Teil des Prototyps. Ob ein Vorschlag fachlich wahr ist, kann das System nicht garantieren.

Damit verbindet die Anforderungsanalyse zwei Ebenen. Fachlich müssen Ziel, Kontext, Verhalten, Kriterien, Evidenz und Unsicherheit sichtbar sein. Technisch müssen Ereignisse authentifiziert sowie Kontext und Modellantwort begrenzt werden. Ergeb-

nisse sind nachvollziehbar zu speichern. Änderungen hängen von einer menschlichen Freigabe ab. Diese Verbindung bildet den roten Faden der Architektur.

4. Konzeption der Systemarchitektur

Eine Person schreibt in einem GitLab-Issue `@dev-assist`. Der Dienst liest das Ticket, antwortet im selben Issue mit Rückfragen oder einem Entwurf und ändert Titel sowie Beschreibung erst nach `@dev-assist publish`. Das Modell darf Inhalte strukturieren. Die Anwendung entscheidet, ob ein Ereignis verarbeitet wird, welcher Kontext das Modell erreicht und ob GitLab verändert wird.

Drei Entwurfsfragen strukturieren das Kapitel. Wie wird der Assistent ausgelöst, ohne GitLab dauernd abzufragen? Wer darf das Issue tatsächlich schreiben? Was darf das Modell allein festlegen? Die Antworten ergeben einen modularen Webhook-Dienst. Das Modell liefert unsichere, sprachliche Vorschläge. Die Anwendung entscheidet nach festen Regeln, ob ein Ereignis verarbeitet wird und ob GitLab verändert wird.

4.1. Systemkontext und Verantwortlichkeiten

Abbildung 4.1 zeigt die beteiligten Systeme und die Übergänge zwischen ihnen. Vier Bereiche sind zu unterscheiden. GitLab verwaltet das verbindliche Issue und erzeugt Ereignisse. Der lokale Dev-Assist-Host enthält die Anwendung, den GitLab-Client und die gespeicherten Kontextdateien. OpenCode bildet eine gesonderte Ausführungsgrenze für den Analyseagenten. Ein konfigurierter Modellanbieter liegt außerhalb des lokalen Hosts.

Jeder rote Pfeil in der Abbildung überschreitet eine Vertrauensgrenze. Ein Webhook darf erst nach Authentifizierung fachlich verarbeitet werden. Eine Modellanfrage darf nur den vorbereiteten Kontext enthalten. Die Antwort gilt als Vorschlag, nicht als Tatsache. Eine produktive Änderung in GitLab darf ausschließlich über die Prozessschicht erfolgen.

Die Architektur verfolgt vier Ziele: klare Verantwortlichkeiten, ereignisbasierte Integration, begrenzte Autonomie und Nachvollziehbarkeit. Getrennte Module erleichtern Tests und Änderungen (vgl. Bass et al., 2021, Kap. 1). Webhooks vermeiden zyklisches Polling (vgl. Hohpe/Woolf, 2003, Abschnitte "Messaging" und "Message Channel"). Ein Vorschau-/Publish-Muster begrenzt den Handlungsspielraum des Modells. Gespeicher-

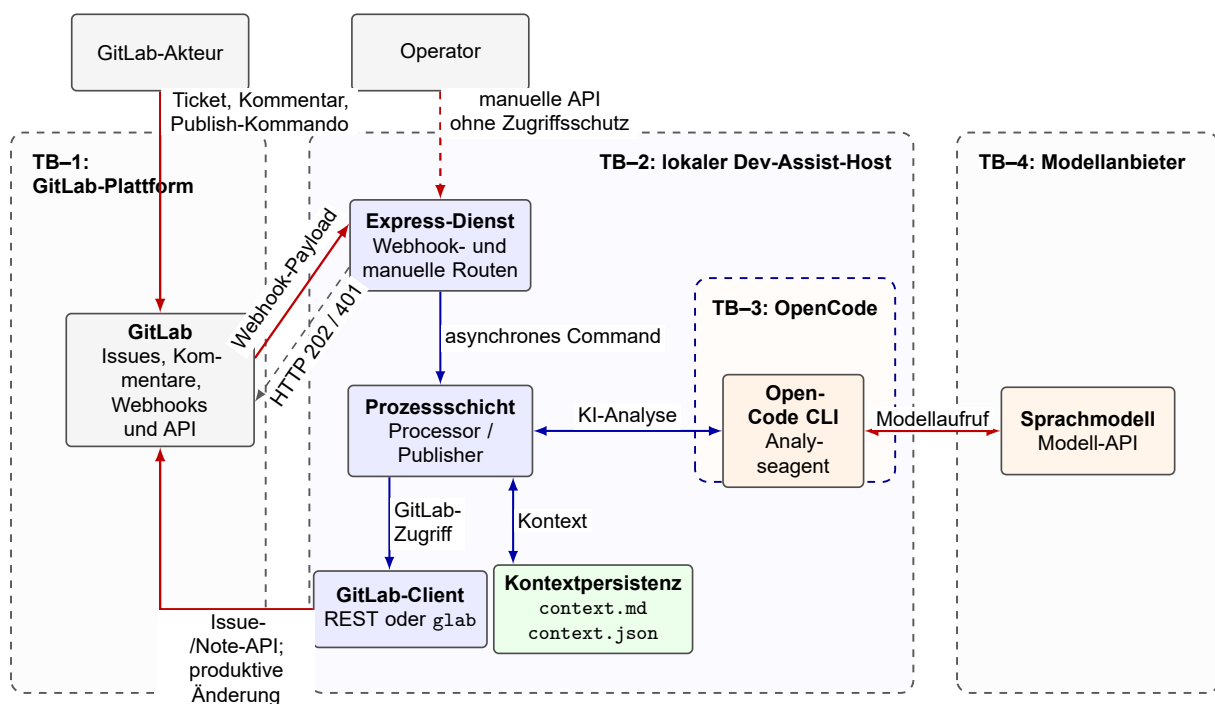


Abbildung 4.1.: Systemkontext und Komponenten des Dev-Assist-Prototyps mit Vertrauensgrenzen. Der Express-Dienst orchestriert GitLab-Zugriffe, KI-Analyse und lokale Kontextpersistenz. Rote Datenflüsse überschreiten eine Plattform-, Prozess- oder Anbietergrenze. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

te Kontextdateien machen den Zwischenstand prüfbar.

Zwei Alternativen wurden verworfen. Ein periodisch abfragender Dienst hätte keine öffentliche Webhook-Grenze, würde aber Verzögerung und zusätzlichen Zustand erzeugen. Ein Agent mit direkten GitLab-Werkzeugen würde Modellentscheidung und produktive Änderung koppeln. Für die Forschungsfrage ist die gewählte Trennung geeigneter: Auslösung, Analyse und Freigabe bleiben getrennt beobachtbar.

Tabelle 4.1 ordnet die Verantwortlichkeiten den später untersuchten Modulen zu. Die Pfade dienen der Nachverfolgbarkeit, nicht als Einstieg.

Tabelle 4.1: Architekturbereiche, Verantwortlichkeiten und zentrale Module

Bereich	Verantwortung	Zentrale Module
HTTP und Webhooks	Annahme, Authentifizierung, Parsing und schnelle Antwort	src/app.ts, src/routes/, src/services/gitlab/auth.ts
GitLab und Kontext	Issues und Notes lesen oder ändern, Bot-Filter, Kommentar-Bereinigung	src/services/gitlab/
Analyse und Ausgabe	Prompt, Mock/OpenCode, Schema, Rückfragen und Markdown	src/services/ai/, opencode.json
Prozess und Persistenz	Analyse orchestrieren, Kontextdateien verwalten und Publish ausführen	src/services/processing/, src/services/context/

4.2. Zentrale Entwurfsentscheidungen

Die erste Entscheidung betrifft die Auslösung. Der Assistent reagiert auf GitLab-Ereignisse, statt Issues zyklisch zu lesen. Das passt zum interaktiven Kommentarverlauf. Es erzeugt zugleich eine öffentliche HTTP-Grenze, an der Authentizität und Wiederholungen zu prüfen sind.

Die zweite Entscheidung trennt Transportbestätigung und Analyse. GitLab erwartet eine schnelle erfolgreiche HTTP-Antwort (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook receiver requirements"). Der Prototyp antwortet mit Status 202: angenom-

men, aber fachlich noch nicht abgeschlossen. Die längere Modellarbeit läuft danach. Eine dauerhafte Warteschlange wäre im Betrieb robuster. Der Prototyp nutzt nur Hintergrundausführung.

Die dritte Entscheidung trennt Analyse und Publish. Eine sofortige Übernahme würde eine fehlerhafte Modellantwort unmittelbar ins Issue schreiben. Deshalb speichert die Anwendung einen Zwischenstand. Publish darf nur diesen Stand übernehmen und keinen neuen Modelllauf auslösen. Der gespeicherte Stand soll genau dem zuvor sichtbaren Vorschlag entsprechen. Kapitel 5 prüft, ob diese Bindung im Ist-Stand gehalten wird.

Die vierte Entscheidung begrenzt Fehler nach ihrer Wirkung. Fehlender Zusatzkontext erlaubt einen gekennzeichneten Fallback. Eine ungültige Modellantwort erzeugt keinen Publish-Kontext. Scheitert die Issue-Aktualisierung, müssen Diskussion und Ausgangsticket erhalten bleiben. Das Löschen der Dev-Assist-Kommentare darf im Soll erst nach bestätigter Kernänderung erfolgen.

Diese Entscheidungen erzeugen Zielkonflikte. Eine frühe HTTP-Antwort vermeidet Timeouts, verlagert fachliche Fehler aber in Logs und Kommentare. Ein striktes Schema erleichtert Tests und kann fachliche Kategorien ausblenden. Zusätzlicher Repository-Kontext erhöht die technische Plausibilität und die Angriffsfläche. Prozesskontrolle hat Vorrang vor einer möglichst autonomen Bedienung.

4.3. Analyse- und Publish-Pfad

Abbildung 4.2 zeigt die beiden Pfade, die eine Person auslöst. Zuerst die Analyse nach `@dev-assist`, danach die Freigabe nach `@dev-assist publish`. Das ist der Sollablauf. Rot markierte Stellen in der Abbildung gehören nicht zum Soll. Kapitel 5 zeigt am Code, wo der untersuchte Stand davon abweicht.

Im Analysepfad lädt die Anwendung nach Möglichkeit das aktuelle Issue und die Notes. Scheitert dieser Abruf, bleibt der Webhook-Text als begrenzter Fallback. Optional ergänzt eine Repository-Zusammenfassung technische Hinweise. Feste Anweisungen und Ticketdaten werden getrennt an den Analyseagenten übergeben. Nur eine schema-konforme JSON-Antwort wird akzeptiert. Je nach offenen Fragen entsteht ein Klärungs-

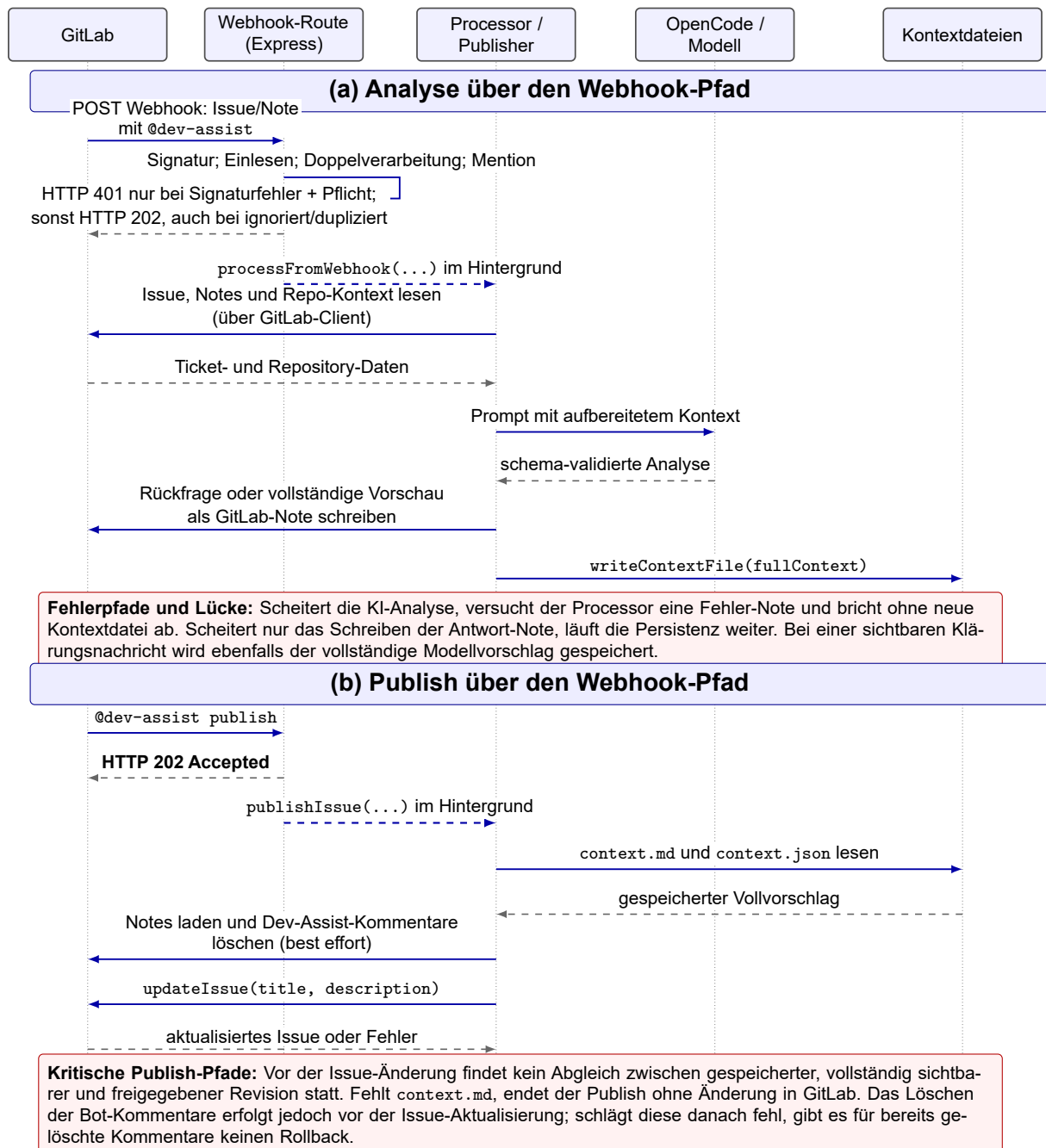


Abbildung 4.2.: Sequenz der Analyse- und Publish-Verarbeitung über den GitLab-Webhook. GitLab erhält sofort eine Bestätigung (HTTP 202); die eigentliche Arbeit läuft danach ohne weitere Rückmeldung an den Absender. Rote Hinweise kennzeichnen Fehlerpfade und dass sichtbare Vorschau, gespeicherter Kontext und Freigabe nicht an dieselbe Fassung gebunden sind. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

kommentar oder ein sichtbarer Entwurf. Parallel wird der Stand lokal gespeichert.

Kommentar und Kontextdatei sind zwei getrennte Wirkungen. Ein sichtbarer Klärungskommentar kann weniger enthalten als die gespeicherte Datei. Genau das ist die Lücke zwischen Vorschau und Freigabe. Publish soll deshalb nur den Stand übernehmen, der vollständig sichtbar war. Im Zielbild aktualisiert der Publisher zuerst das Issue und löscht Dev-Assist-Kommentare erst danach. Das Modell wird in diesem Pfad nicht erneut aufgerufen.

Tabelle 4.2 fasst die vorgesehene Reaktion auf typische Fehler zusammen.

Tabelle 4.2: Fehlerklassen und vorgesehene Reaktionen im Zielablauf

Fehlerklasse	Systemreaktion	Begründung
Ungültiger Webhook	Ablehnung vor fachlicher Verarbeitung	Externe Eingänge dürfen keine internen Schritte auslösen.
Fehlender Zusatzkontext	Warnung und begrenzter Fallback	Ein Vorschlag kann möglich bleiben, muss aber Kontextlücken erkennen lassen.
Ungültige Modellantwort	Kein normaler Kontext; Fehler wird sichtbar gemacht	Strukturell unkontrollierter Modelltext darf nicht in einen Publish gelangen.
Fehlgeschlagenes Issue-Update	Publish-Abbruch, Kommentare bleiben erhalten	Die bestehende Diskussion darf nicht vor der dauerhaften Änderung verloren gehen.
Fehler beim Löschen der Dev-Assist-Kommentare	Protokollierung nach erfolgreichem Update	Die Kernänderung ist bereits erfolgt; das Löschen bleibt nachgeordnet.

4.4. Was das Modell ausgehen darf

Der Prompt trennt feste Regeln von nicht vertrauenswürdigen Ticketdaten. Er fordert Ziel, Nutzen, Beschreibung, Akzeptanzkriterien, belegten technischen Kontext, Lösungsvorschlag und offene Fragen. Repository-Hinweise dürfen Technik stützen, aber keine fachlichen Anforderungen erfinden.

Das Ausgabeschema enthält genau sechs Felder: Titel, Beschreibung, Akzeptanzkriterien, technischen Kontext, Lösungsvorschlag und offene Fragen. Das Format ist leichter zu prüfen als freier Fließtext. Es bildet das breitere fachliche Zielmodell aus Kapitel 3 jedoch nicht vollständig ab. Quellenbasis, Risiken und Validierungsschritte haben keine eigenen Felder. Diese Differenz bleibt in der Evaluation sichtbar.

Bilder und Screenshots werden nicht heruntergeladen oder inhaltlich analysiert. Die Repository-Zusammenfassung bleibt auf Metadaten, Sprachen, einen Dateibaum-Ausschnitt und wenige Schlüsseldateien begrenzt.

4.5. Sicherheits- und Testbarkeitsziel

Die Architektur fordert, dass GitLab jede Lieferung mit einem geheimen Schlüssel signiert und die Anwendung diese Signatur sowie den Zeitstempel prüft. So zählen gefälschte oder alte Anfragen nicht (vgl. GitLab Documentation, o. J.a, Abschnitt "Signing tokens"). Die Signaturpflicht soll standardmäßig aktiv sein. Ein toleranter Modus ist nur für lokale Entwicklung vorgesehen. Manuelle Process- und Publish-Wege brauchen einen eigenen Zugangsschutz, weil sie dieselbe Logik ohne Webhook durchlaufen.

Zusätzlich sollen die Aktivierungsregel, das Ignorieren eigener Bot-Kommentare und das Unterdrücken kurzfristiger Doppelverarbeitung unbeabsichtigte Läufe verhindern. Projekt- und Issue-Kennungen müssen vor der Speicherung auf erlaubte Werte begrenzt werden. Ein Mock-Provider macht Parser, Schema und Formatter ohne Modellkosten prüfbar. Kapitel 5 zeigt, welche dieser Grenzen der Prototyp hält und welche er verfehlt.

4.6. Zwischenfazit

Die Architektur beantwortet die drei Entwurfsfragen durch Webhooks, getrennte Freigabe und ein festes Ausgabeschema. Kontrolle entsteht außerhalb des Modells. Kapitel 5 zeigt, wie dieser Entwurf im TypeScript-Prototyp umgesetzt ist und wo der Ist-Stand vom Zielbild abweicht.

5. Implementierung

Dev-Assist besitzt keine eigene Oberfläche. Die Bedienung liegt vollständig in GitLab. Dieses Kapitel erklärt den Prototyp deshalb von dem, was eine Person im Issue sieht, zum Code, der genau diese Schritte umsetzt. Zuerst der sichtbare Ablauf, dann der Leistungsstand, dann vier Stellen, an denen Soll und Ist auseinanderlaufen. Eine vollständige Datei-zu-Datei-Beschreibung ist nicht das Ziel.

5.1. Was eine Person im Issue sieht

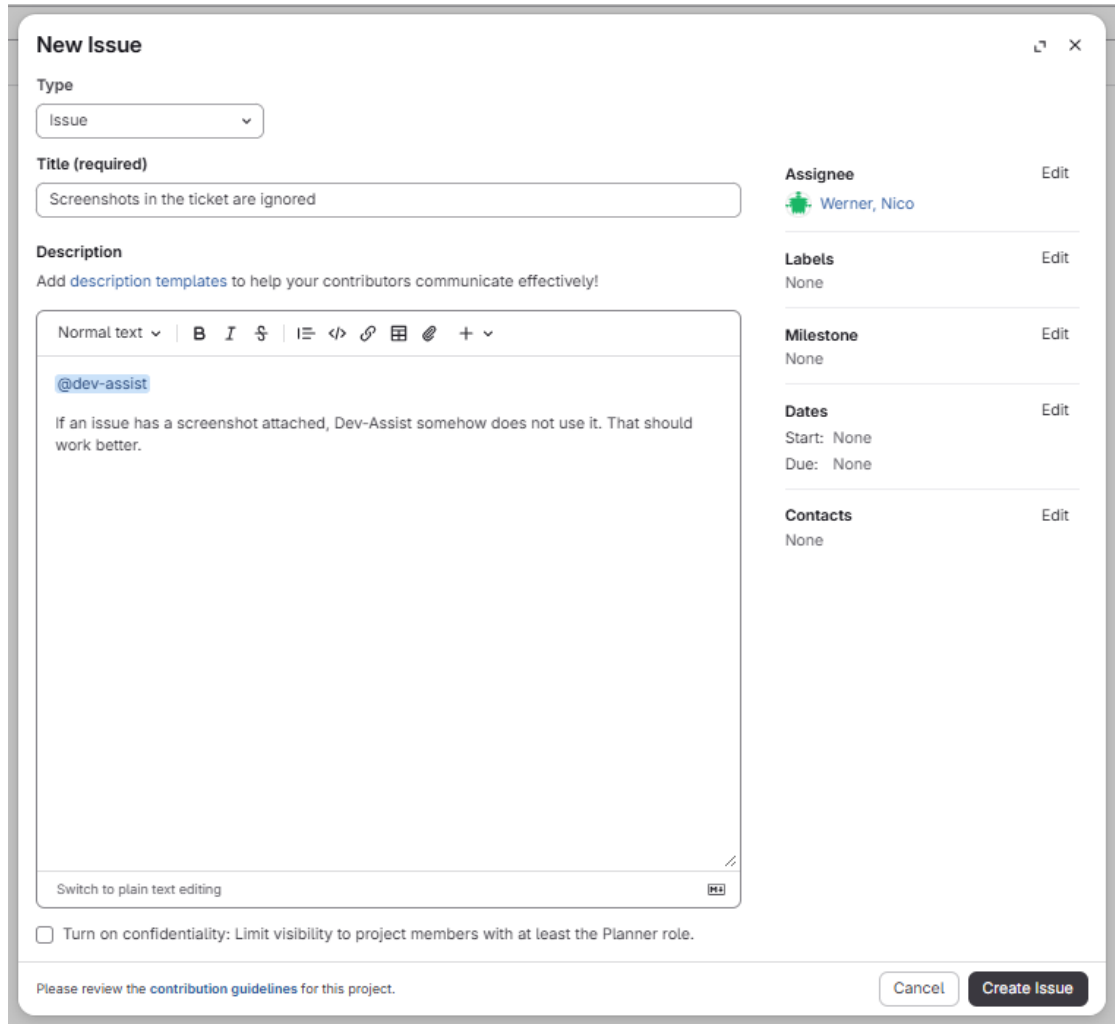
Die folgenden Screenshots stammen aus einem dokumentierten GitLab-Lauf im untersuchten Dev-Assist-Projekt. Das Beispielissue betrifft eine Lücke des Prototyps selbst: angehängte Screenshots werden bei der Analyse nicht gelesen. Es handelt sich um ein eigens angelegtes Demonstrationsissue, nicht um betriebliche Originaldaten und nicht um einen Wirksamkeitsnachweis.

Abbildung 5.1 zeigt den Einstieg. Titel und Beschreibung bleiben absichtlich unvollständig. Die erste inhaltliche Zeile enthält `@dev-assist`.

Abbildung 5.2 zeigt die Antwort im selben Issue. Der Dienst stellt zwei fachliche Rückfragen zu Umfang und sichtbarer Verwendung der Screenshots. Die meldende Person beantwortet sie in einem weiteren Kommentar, wiederum mit führendem `@dev-assist`.

Nach ausreichendem Kontext erscheint ein strukturierter Vorschlag als Kommentar. Erst `@dev-assist publish` schreibt den gespeicherten Stand in Titel und Beschreibung. Abbildung 5.3 zeigt das Issue danach. Die Dev-Assist-Kommentare sind entfernt. Der ausführliche Verlauf steht in Anhang A.

Tabelle 5.1 übersetzt denselben Ablauf in den technischen Leistungsstand und trennt vorhandene Kernfunktionen von den später bewerteten Abweichungen.



New Issue

Type
Issue

Title (required)
Screenshots in the ticket are ignored

Description
Add [description templates](#) to help your contributors communicate effectively!

Normal text | **B** *I* U | +

@dev-assist

If an issue has a screenshot attached, Dev-Assist somehow does not use it. That should work better.

Switch to plain text editing

☐ Turn on confidentiality: Limit visibility to project members with at least the Planner role.

Please review the [contribution guidelines](#) for this project.

Assignee: Werner, Nico Edit

Labels: None Edit

Milestone: None Edit

Dates: Start: None Due: None Edit

Contacts: None Edit

Cancel Create Issue

Abbildung 5.1.: Anlegen eines unvollständigen Dev-Assist-Issues in GitLab. Die erste inhaltliche Zeile enthält @dev-assist. Der Screenshot stammt aus einem dokumentierten Lauf im untersuchten Projekt.



dev-assist
@project_599_bot_40f728b5658b352de44dd525734c4cf9
just now

Maintainer






Dev-Assist: More information needed

I looked at the issue, but there is not enough detail yet to create a solid, developer-ready ticket.

Important: I only ask about requirements, user needs, acceptance criteria, scope and success criteria. I do **not** need details about the current codebase, tech stack, specific components or existing implementation — the developer will handle those.

Please reply with answers to these questions (mention `@dev-assist` again with the details):

- Should screenshot support apply only to attachments on the main issue, or also to images added later in comments?
- Should Dev-Assist surface screenshot findings explicitly in the generated ticket text, or use them only internally to improve analysis quality?

Once I have the missing pieces I will post a full structured proposal and you can use `@dev-assist publish` to apply it.

(Current best guess for title: Use issue screenshots during Dev-Assist analysis)



Werner, Nico @nico.werner
just now

Author Maintainer






`@dev-assist`

- Should screenshot support apply only to attachments on the main issue, or also to images added later in comments?
 - Screenshot support should apply to attachments and comments
- Should Dev-Assist surface screenshot findings explicitly in the generated ticket text, or use them only internally to improve analysis quality?
 - The screenshots should surface in the generated ticket

Abbildung 5.2.: Klärungskommentar von Dev-Assist und anschließende Antworten im selben Issue. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.

Use issue screenshots in Dev-Assist analysis

  Issue created 6 minutes ago by Werner, Nico

Description

Dev-Assist should include screenshots from both the main issue and later comments when analyzing a GitLab ticket so that visual information is not ignored during ticket generation. This should improve the completeness and accuracy of the generated developer-ready ticket when key requirements, errors, UI states, or workflows are shown only in attached images.

The generated ticket should explicitly surface relevant findings from screenshots instead of using them only internally. Scope includes collecting screenshots attached to the issue description and comments, incorporating their content into the analysis workflow, and reflecting meaningful visual findings in the resulting structured ticket text when they add functional or contextual value.

Acceptance Criteria

- When an issue contains one or more screenshots in the main issue content, Dev-Assist uses them during analysis instead of ignoring them.
- When screenshots are added in issue comments, Dev-Assist also includes them in the analysis scope.
- The generated structured ticket explicitly reflects relevant information derived from screenshots when that information affects the described problem, requirements, scope, or acceptance criteria.
- If both text and screenshots are present, the output combines both sources into a coherent ticket rather than relying on text alone.
- If no screenshots are present, the existing text-only analysis flow continues to work without regression.
- Automated or manual validation demonstrates that issues with screenshot-only or screenshot-heavy context produce materially more complete ticket output than before.

Technical Context & Logs

- The current issue reports that Dev-Assist does not use screenshots attached to GitLab issues.
- Prior clarification confirms that screenshot support must apply to attachments on the main issue and to images added later in comments.
- Prior clarification confirms that screenshot findings should surface in the generated ticket text.
- The repository summary shows that GitLab issue processing is coordinated through `src/services/processing/*`, AI integration lives under `src/services/ai/*`, and GitLab access/parsing lives under `src/services/gitlab/*`.
- The repository summary states that Dev-Assist processes GitLab issue and note webhook content and uses the `opencode` CLI with the `dev-assist-analyzer` agent for real analysis.
- Assumption: screenshot handling may depend on whether the active AI/runtime path can receive image inputs or extracted image context in addition to text.

Proposed Solution

- Extend issue and comment context collection so screenshot attachments and embedded images from the main issue and subsequent comments are detected and included in the analysis input.
- Normalize the collected screenshot references and associated metadata into the issue context passed through the processing pipeline.
- Update the AI analysis flow so screenshot content is available alongside textual issue content, using direct image-capable input when supported or a fallback representation when necessary.
- Adjust the analysis instructions or formatting logic so relevant screenshot findings are explicitly reflected in the generated ticket description, acceptance criteria, or technical context when applicable.
- Add automated tests covering screenshots on the main issue, screenshots in comments, mixed text-plus-image issues, and the no-screenshot path to prevent regressions.

Edited just now by dev-assist



 Add design  Create merge request





Assignee	Edit
 Werner, Nico	
Labels	Edit
None	
Milestone	Edit
None	
Dates	Edit
Start: None	
Due: None	
Time tracking	+
Add an estimate or time spent.	
2 Participants	
 	

Abbildung 5.3.: Dasselbe Issue nach `@dev-assist publish`. Titel und Beschreibung enthalten den gespeicherten Entwurf. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.

Tabelle 5.1: Leistungsstand der Kernfunktionen im untersuchten Prototyp

Kernfunktion	Umgesetzter Stand
Webhook-Annahme	Issue- und Note-Ereignisse werden geparkt; akzeptierte Lieferungen erhalten HTTP 202, die Analyse läuft danach.
Kontextaufbereitung	Aktuelles Issue, Notes und optional begrenzter Repository-Kontext werden vor der Analyse zusammengeführt.
Modellgrenze	OpenCode oder Mock liefern eine Antwort; nur das JSON-Schema mit sechs Feldern wird akzeptiert.
Rückfrage oder Vorschlag	Mindestens zwei substantielle offene Fragen erzeugen einen Klärungskommentar, sonst einen strukturierten Entwurf.
Speicherung und Publish	Der analysierte Stand wird lokal gespeichert; @dev-assist publish übernimmt ihn ohne neuen Modelllauf.
Nachweisbare Teilprüfungen	Parser, Schema, Formatter und ausgewählte Routen sind durch 60 Tests abgesichert.

Dieser Kern ist lauffähig und lokal ohne reales Modell testbar. Er erfüllt noch nicht den vollständigen Kontrollanspruch aus Kapitel 3. Die folgenden Abschnitte zeigen das an konkreten Code-Stellen statt an einer vollständigen Modulbeschreibung.

5.2. Projektaufbau

Tabelle 5.2 gibt die Übersicht. Der Dienst ist eine Node.js-/TypeScript-Anwendung. `npm run dev` startet den Entwicklungsserver, `npm run build` kompiliert nach `dist/`, `npm test` führt die Testsuite aus. Node.js stellt die Laufzeit bereit, Express das Routing, TypeScript die statische Prüfung interner Datenformen (vgl. Node.js Documentation, o. J., o. S.; Express Documentation, o. J., o. S.; TypeScript Documentation, o. J., o. S.). Externe Lieferungen und Modellantworten prüft TypeScript zur Laufzeit nicht. Deshalb bleiben Parsing und Schema-Validierung erforderlich.

Tabelle 5.2: Projektbereiche und umgesetzte Verantwortlichkeiten

Projektbereich	Umgesetzte Verantwortung
<code>src/routes/</code>	Health-Check, Webhook-Eingang sowie derzeit ungeschützte manuelle Process-/Publish-Endpunkte
<code>src/services/gitlab/</code>	Signatur, Einlesen der Lieferung, Mention, Kommando, GitLab-Client und Löschen der Dev-Assist-Kommentare
<code>src/services/ai/</code>	Prompt, Mock-/OpenCode-Aufruf, Schema, Rückfragen und Markdown-Formatierung
<code>src/services/context/</code> , <code>src/services/processing/</code>	Persistenz sowie Orchestrierung der Analyse- und Publish-Phase
<code>tests/</code>	Automatisierte Prüfung ausgewählter fester Programmteile und Grenzfälle

`src/server.ts` startet die Anwendung. `src/app.ts` registriert Middleware und Routen. Die OpenCode-Agenten liegen in `opencode.json`. `AI_PROVIDER=mock` erlaubt einen wiederholbaren Betrieb ohne externes Modell. GitLab-Zugriffe erfolgen über `glab` oder `REST`. `GITLAB_REQUIRE_SIGNATURE` ist standardmäßig `false`. Fehlt das Geheimnis für die Signaturprüfung, akzeptiert die Prüffunktion Webhooks auch dann, wenn das Flag aktiviert ist.

5.3. Vier Stellen, an denen Soll und Ist auseinanderlaufen

Die folgenden Listings zeigen Entscheidungen, die eine Person im Issue merkt oder die eine Freigabe unsicher machen. Sie ersetzen keine vollständige Implementierungsbeschreibung.

5.3.1. Zu weite Aktivierung

FA-1 verlangt `@dev-assist` am Beginn der ersten inhaltlichen Zeile. Listing 5.1 sucht die Erwähnung im gesamten Text. Bei Issue-Ereignissen genügt sie außerdem im Titel. Ein beiläufiges `@dev-assist` mitten in einer Diskussion startet deshalb bereits eine Analyse. Die Bewertung gegen FA-1 folgt in Kapitel 6. Hier ist nur der Ist-Stand am Code zu

sehen.

```
function hasMention(text: string | undefined | null): boolean {  
  const t = String(text || '').trim();  
  if (!t) return false;  
  return mentionPattern.test(t);  
}
```

Listing 5.1: Aktivierung: die Mention wird im gesamten Text gesucht, nicht nur am Zeilenanfang (`mention.ts`)

5.3.2. Strukturelle, nicht inhaltliche Modellgrenze

Listing 5.2 akzeptiert nur sechs Felder. Unbekannte Felder oder falsche Typen führen zum Abbruch. Die Prüfung ist streng gegenüber der Struktur. Sie sagt nichts darüber, ob ein Akzeptanzkriterium fachlich stimmt. Ob der Inhalt fachlich stimmt, ist damit nicht gezeigt. Diese inhaltliche Prüfung ist nicht Teil der Implementierung.

```
const REQUIRED_FIELDS = [  
  'title',  
  'description',  
  'acceptanceCriteria',  
  'technicalContext',  
  'proposedSolution',  
  'openQuestions',  
] as const;
```

Listing 5.2: Ausgabeschema der Modellantwort (`schema.ts`)

5.3.3. Sichtbare Rückfrage, gespeicherter Vollvorschlag

Abbildung 5.2 zeigt nur die Rückfragen und die späteren Antworten. Listing 5.3 erklärt, warum trotzdem mehr gespeichert wird. Der Processor zählt offene Fragen mit mehr als fünf Zeichen. Ab zwei solchen Fragen erscheint ein Klärungskommentar. Unabhängig davon wird der vollständige Analysekontext lokal geschrieben. Eine Person kann also Rückfragen sehen und später einen umfangreicheren Stand veröffentlichen, den sie nie vollständig bestätigt hat.

```
const openQs = (analysis.openQuestions || [])
  .filter((q) => q && q.trim().length > 5);
const needsClarification = openQs.length >= 2;

if (needsClarification) {
  commentToPost = renderClarificationComment(analysis);
} else {
  commentToPost = fullContext;
}
await gitlab.createNote(projectId, issueId, commentToPost);
await writeContextFile(projectId, issueId, fullContext, metadata);
```

Listing 5.3: Klärungsschwelle: ab zwei offenen Fragen erscheint nur der Klärungskommentar, gespeichert wird trotzdem der Vollvorschlag (`processor.ts`)

Die Schwelle berücksichtigt nicht, ob eine einzelne Frage fachlich kritisch ist. Eine Sicherheitsentscheidung kann blockieren. Mehrere Randdetails müssen es nicht.

5.3.4. Falsche Publish-Reihenfolge

FA-11 verlangt zuerst die Issue-Aktualisierung und erst danach das Löschen der Dev-Assist-Kommentare. Listing 5.4 kehrt das um. Schlägt `updateIssue` fehl, können Rückfragen und Entwurf bereits entfernt sein. Genau dann fehlt der Diskussionsverlauf, den Abbildung 5.2 noch zeigt.

```
const notes = await gitlab.listNotes(projectId, issueId);
const deletable = filterDeletableNotes(notes);
for (const n of deletable) {
  await gitlab.deleteNote(projectId, issueId, n.id);
}
await gitlab.updateIssue(projectId, issueId, baseUpdates);
```

Listing 5.4: Ist-Stand der Publish-Reihenfolge: Kommentare werden vor der Issue-Aktualisierung gelöscht (`publisher.ts`)

5.4. Weitere Grenzen, knapp

Vier weitere Abweichungen gehören zur späteren Bewertung, brauchen aber keine vollständige Methodenliste. Die Signaturprüfung weicht vom von GitLab vorgesehenen Verfahren ab und prüft den Zeitstempel nicht; ohne Geheimnis gilt ein Webhook als gültig. Die interne Ereignisdarstellung enthält keinen Autor, FA-2 ist daher nur teilweise erfüllt. Manuelle HTTP-Routen und der CLI-Publish verwenden dieselbe Prozesslogik ohne eigenen Zugangsschutz und ohne Prüfung der Projekt- und Issue-Kennungen. Speicherpfade werden aus diesen Kennungen gebildet, ohne sie auf positive Ganzzahlen zu begrenzen.

Bilder werden nicht analysiert. Die Repository-Zusammenfassung bleibt auf Metadaten, Sprachen und wenige Dateien begrenzt.

5.5. Was die Tests tatsächlich tragen

Der Mock-Provider und kleine Parser-, Schema- und Formatterfunktionen ermöglichen lokale Tests ohne Modellkosten. Die 60 bestandenen Tests belegen vor allem diese Teilfunktionen. Zeitstempelprüfung, Routenautorisierung, Pfadbegrenzung, Publish-Reihenfolge und der vollständige Weg von Webhook bis Issue-Änderung bleiben ungetestet.

Die Implementierung belegt damit den Kern aus Tabelle 5.1: Die Modellantwort wird strukturell begrenzt, Analyse und Publish sind getrennt, die Bedienung bleibt in GitLab. Die offenen Muss-Anforderungen an Aktivierung, Authentifizierung, Pfade und Publish-Reihenfolge bewertet Kapitel 6.

6. Qualitätssicherung und Evaluation

Dieses Kapitel prüft, ob der vorhandene Prototyp das Soll aus Kapitel 3 einhält. Dafür gibt es drei Quellen: einen dokumentierten Build- und Testlauf, das Nachverfolgen der entscheidenden Programmwege und einen Abgleich mit der GitLab-Dokumentation. Ein erfolgreicher Build oder ein bestandener Test gilt nur für den konkret geprüften Fall. Er belegt weder den gesamten Ablauf noch die fachliche Qualität realer Modellantworten.

Die Untersuchung ist kein Experiment mit Nutzenden und keine Feldstudie. Die Methode wird deshalb zusammen mit ihren Aussagegrenzen dargestellt (vgl. Easterbrook et al., 2008, S. 285 f. und S. 302 f.).

6.1. Evaluationsfragen

Die Forschungsfragen betreffen den Informationsbedarf von Tickets und die kontrollierte Einbindung in GitLab. Für die technische Prüfung werden vier engere Fragen verwendet. Sie sind keine zusätzlichen Forschungsfragen. Sie legen nur fest, welche Aussage aus welcher Evidenz folgen darf:

1. **EF-1 – Reproduzierbarkeit:** Lässt sich der festgehaltene Programmstand mit den dokumentierten Werkzeugen bauen, und laufen die vorhandenen Tests erfolgreich und wiederholbar?
2. **EF-2 – funktionale Übereinstimmung:** Welche Schritte von Aktivierung, Kontext, Analyse, Klärung, Vorschau, Speicherung und Publish sind im Code vorhanden und durch passende Prüfungen belegt?
3. **EF-3 – Sicherheits- und Integritätsgrenzen:** Stimmen Echtheitsprüfung, Eingabeprüfung, Rechte und die Reihenfolge dauerhafter Änderungen mit den Muss-Anforderungen und der GitLab-Dokumentation überein?
4. **EF-4 – Nachweis der Modellwirkung:** Was lässt sich mit den vorhandenen Daten über Richtigkeit, Vollständigkeit, Qualität der Rückfragen und praktischen

Nutzen sagen?

Bewertet wird jeweils ein vollständiger Schritt, nicht eine einzelne Datei. Bei der Ereignisannahme gehören dazu die Eingangsdaten, die Prüfung der Echtheit, das Einlesen und die Frage, ob das Ereignis relevant ist. Bei Publish gehören dazu gespeicherter Kontext, sichtbarer Vorschlag, Freigabe, Issue-Änderung und das Löschen der Dev-Assist-Kommentare. Eine isoliert korrekte Funktion darf nicht für einen unvollständigen Gesamtprozess stehen.

Die Fragen trennen drei Ebenen. Erstens: Welche Dateien, Konfigurationen und Tests liegen vor? Zweitens: Welche Entscheidung trifft ein Programmweg bei konkreten Eingaben? Drittens: Welche Schlussfolgerung ist daraus zulässig? Ein bestandener Einlese-Test kann belegen, dass eine Anfrage in das interne Format überführt wird. Er belegt weder, dass eine reale Lieferung echt war, noch dass der spätere Vorschlag fachlich stimmt. Diese Begrenzung gilt für das ganze Kapitel.

6.2. Untersuchungsstand und Artefakte

Die Prüfung wurde am 31.07.2026 unter Windows mit Node.js v24.11.0 und npm 11.6.1 durchgeführt. Der untersuchte Programmstand ist der Git-Commit d179eb22beb0a59006869a3acc7829f3070fc678. Die Textänderungen dieser Arbeit gehören nicht zum geprüften Quellcode. Gebaut und getestet wurde mit `npm run build` und `npm test`.

Tabelle 6.1: Ergebnisse des dokumentierten Build- und Testlaufs

Prüfgröße	Ergebnis
TypeScript-Build	Erfolgreich; keine TypeScript-Fehler
Automatisierte Tests	60 von 60 Tests in 15 Testsuiten bestanden
Teststatus und Laufzeit	Keine fehlgeschlagenen, übersprungenen oder offenen Tests; ca. 0,6 Sekunden
Testabdeckung	Nicht erhoben; Skript und Konfiguration zur Testabdeckung fehlen
Kontinuierliche Integration	Keine GitHub- oder GitLab-CI-Konfiguration im Repository

Tabelle 6.1 dokumentiert einen einzelnen erfolgreichen Lauf. Die Tests prüfen vor allem feste Programmteile: Einlesen, Schema, Darstellung, ausgewählte Laufzeitpfade und Aufbereitung. Es gibt keinen Test, der einen echten GitLab-Webhook über Modellaufruf und Speicherung bis zur Issue-Änderung durchläuft. Die Testzahl darf deshalb weder als Abdeckung noch als Zuverlässigkeitskennzahl gelesen werden.

Statt einer langen Dateiliste fasst Tabelle 6.2 die geprüften Quellen nach ihrer Funktion zusammen (vgl. Werner, 2026). Version, Standdatum und Bezug der Evidenz bleiben dadurch nachvollziehbar.

Tabelle 6.2: Untersuchte projektinterne Artefaktgruppen

Artefaktgruppe	Umfang	Wozu sie dient
Implementierung	src/, opencode.json	Programmwege und Grenzen zwischen Systemen
Testbasis	15 Dateien unter tests/	ausgeführte Fälle mit festgelegten Eingaben
Build und Konfiguration	package.json, tsconfig.json, .env.example	Befehle, Versionen und Sicherheitsstandardwerte
Ziel- und Betriebsbeschreibung	README.md, descriptions/ project_description.txt, AGENTS.md	Sollprozess, Nutzung und Projektkontext

6.3. Prüfprotokoll und Nachverfolgbarkeit

Die Prüfung folgt einer festen Reihenfolge. Zuerst werden Commit, Datum und Laufzeitumgebung festgehalten. Anschließend laufen Build und die vollständige Testsuite. Einzelne Tests werden nicht ausgelassen. Danach wird gelesen, welches Verhalten die Tests als richtig festlegen. Ein bestandener Test reicht nicht. Maßgeblich ist, was die Prüfung tatsächlich verlangt. Für jede Muss-Anforderung wird der Weg von der Eingabe bis zu allen sichtbaren Wirkungen verfolgt. Die Signaturvorgaben von GitLab werden zuletzt mit der zum Prüfdatum geltenden Dokumentation verglichen.

Tabelle 6.3 macht dieses Vorgehen und die jeweilige Grenze nachvollziehbar. Die Schritte ergänzen einander. Wer nur den Quellcode liest, kann einen geplanten Zweig erkennen, aber nicht belegen, dass er ausführbar ist. Ein Test kann ein Verhalten ausführen und zugleich eine falsche Annahme über GitLab als richtig festschreiben. Erst der Abgleich erlaubt eine begrenzte Aussage zur Übereinstimmung mit dem Soll.

Tabelle 6.3: Prüfprotokoll und zulässige Schlussfolgerungen

Prüfschritt	Erhobener Beleg	Was daraus folgen darf
Stand fixieren	Commit, Standdatum, Plattform und Werkzeugversionen	Spätere Textänderungen dürfen nicht als geprüfte Implementierung erscheinen
Build ausführen	Erfolgreiche TypeScript-Übersetzung oder konkreter Fehler	Belegt Übersetzbarkeit, nicht Laufzeit- oder Fachverhalten
Tests ausführen und Erwartungen lesen	Teststatus, Suiten, Eingaben und Assertions	Belegt nur die ausgeführten Fälle und das im Test erwartete Verhalten
Programmweg verfolgen	Schutzprüfungen, Ausweichwege, Speicherung und Reihenfolge der Wirkungen	Belegt vorhandene Wege; nicht erreichbare externe Zustände werden nicht simuliert
Vorgaben abgleichen	Sollanforderung und GitLab-Dokumentation	Interne Stimmigkeit gilt nur bei Übereinstimmung mit den maßgeblichen Vorgaben
Nachweislücke protokollieren	Fehlender Test, fehlende Messung oder fehlende Betriebsdaten	Fehlende Evidenz heißt <i>nicht untersucht</i> , nicht automatisch Fehlfunktion

Jeder Befund hat vier Teile: die betroffene Anforderung, der beobachtete Ist-Pfad, die zugehörige Evidenz und die Begründung des Status. Das ist besonders bei negativen Ergebnissen wichtig. Wird eine Signaturprüfung als nicht erfüllt bewertet, reicht weder der Hinweis auf eine fehlende Testdatei noch eine allgemeine Sicherheitsannahme. Nötig ist der Vergleich aus gefordertem Nachrichtenformat, implementierter Berechnung, Standardwerten und dem Umgang mit fehlendem Geheimnis. Die Publish-Reihenfolge wird an der beobachtbaren Aufrufreihenfolge und ihrer Fehlerwirkung beurteilt. Der Funktionsname oder die beabsichtigte Architektur reichen nicht aus.

Die Tests wurden nicht nachträglich an das Soll angepasst. Ihre unveränderte Ausführung beantwortet EF-1 und liefert für EF-2 ausgeführte Fälle. Weicht ein bestandener Test vom Soll ab, ist das ein eigener Befund. Die Implementierung darf ihre Übereinstimmung nicht allein über selbst gesetzte Erwartungen belegen.

6.4. Bewertungslogik

Geprüft werden die Stellen, an denen Daten eine Grenze überschreiten: vom Webhook zum internen Ereignis, vom Ticketkontext zum Modelleingabetext, von der Modellantwort zum geprüften Objekt und vom gespeicherten Vorschlag zur Issue-Änderung in GitLab. Für jeden Übergang gibt es drei Arten von Belegen. Ein ausgeführter Test bestätigt einen Fall mit festen Eingaben. Das Lesen des Codes zeigt, welcher Weg im Programm vorhanden ist. Der Abgleich mit Anforderung oder GitLab-Dokumentation zeigt, ob dieser Weg dem Soll entspricht.

Eine Anforderung gilt als *nachgewiesen*, wenn Soll und Ist übereinstimmen und passende Belege vorliegen. *Teilweise nachgewiesen* heißt: Bausteine sind da, der Gesamtschritt oder das Zielverhalten bleibt unvollständig. *Nicht erfüllt* gilt, wenn die Implementierung der Muss-Anforderung widerspricht, auch wenn ein Test genau dieses abweichende Verhalten bestätigt. *Nicht untersucht* gilt für fachliche Wirkung oder Betrieb, zu denen keine geeigneten Daten vorliegen. So wird interne Stimmigkeit nicht mit äußerer Übereinstimmung verwechselt.

Sicherheits- und Änderungsgrenzen wiegen schwerer als reine Darstellungsfragen. Eine falsche Überschrift ist korrigierbar. Eine unberechtigte Verarbeitung oder eine vorzeitig gelöschte Diskussion kann den Ticketstand dagegen unbrauchbar machen. Die Bewertung folgt deshalb der Kette Anforderung, Implementierung, Beleg und begrenzte Interpretation.

6.4.1. Prüfung der Sicherheits- und Freigabegrenzen

Für EF-3 reicht es nicht, einzelne Schutzfunktionen zu finden. Eine Sicherheitsgrenze wirkt nur, wenn sie an jedem erreichbaren Eingang gilt. Im Fehlerfall muss die Verarbeitung abgelehnt werden. Spätere Schritte dürfen diese Entscheidung nicht umgehen.

Deshalb gelten drei Regeln:

Ereignisregel: Nur verarbeiten, wenn die Lieferung echt, aktuell und fachlich relevant ist.

Freigaberegeln: Nur veröffentlichen, wenn genau diese Fassung vollständig sichtbar und ausdrücklich freigegeben war.

Änderungsregel: Kommentare erst löschen, wenn die Issue-Aktualisierung erfolgreich war.

Die Ereignisregel verbindet Herkunft, Aktualität und Relevanz. Eine korrekte Signaturprüfung allein genügt nicht, wenn alte Lieferungen unbegrenzt akzeptiert werden. Umgekehrt ersetzt eine präzise Erkennung von @dev-assist keine Echtheitsprüfung. Für manuelle Process- und Publish-Wege ist ein gleichwertiger Zugangsschutz nötig, auch wenn dort kein GitLab-Webhook ankommt. Die Prüfung betrachtet daher alle Eingänge, die dieselbe Analyse- oder Freigabelogik erreichen.

Die Freigaberegeln bindet die menschliche Entscheidung an genau die Fassung, die sichtbar war. Die zeitliche Trennung zwischen Analyse und Publish ist dafür notwendig, aber nicht hinreichend. Ohne Versionskennung kann ein gespeicherter Inhalt nach der Sichtprüfung verändert werden. Er kann bereits während einer Klärung umfangreicher sein als der sichtbare Kommentar. Ein Publish-Kommando allein reicht deshalb nicht. Geprüft wird, ob vor der Issue-Änderung dieselbe Fassung sichtbar, identifizierbar und freigegeben war.

Die Änderungsregel ordnet Wirkungen, die sich nicht oder nur schwer rückgängig machen lassen. Die Issue-Aktualisierung ist die fachliche Kernänderung. Das Löschen der Dev-Assist-Kommentare kommt danach. Wird die Reihenfolge umgedreht, kann ein Fehler einen widersprüchlichen Zustand erzeugen. Dann fehlen sowohl das alte Ticket mit vollständiger Diskussion als auch der neue Stand. Die sichere Reihenfolge muss im Programmweg stehen. Ein Test, in dem die Issue-Aktualisierung fehlschlägt, muss sie absichern.

Zusätzlich werden Eingaben und Wiederholungen geprüft. Projekt- und Issue-Kennungen sind nicht nur Parameter der Schnittstelle, sondern Teile lokaler Speicherpfade. Ihr zulässiger Wertebereich muss feststehen, bevor ein Pfad gebildet wird. Damit dieselbe Lieferung nicht zweimal verarbeitet wird, braucht jede Lieferung eine stabile Identität. Laufen mehrere Instanzen, müssen sie denselben Stand teilen.

Ein nur im Arbeitsspeicher gehaltener Schlüssel kann kurzfristige Wiederholungen begrenzen. Er erfüllt aber keine allgemeine Garantie, dass jede Lieferung genau einmal verarbeitet wird. Die Bewertung behauptet das nicht.

6.4.2. Trennung technischer und inhaltlicher Prüfung

EF-4 braucht andere Daten als EF-1 bis EF-3. Die Schema-Prüfung misst, ob eine Antwort die erwarteten Felder und Datentypen besitzt. Sie misst nicht, ob ein vorgeschlagenes Akzeptanzkriterium aus dem Ticket folgt. Ebenso wenig misst sie, ob eine Rückfrage die wichtigste Lücke trifft. Deshalb bleiben drei Ebenen getrennt: die maschinelle Verarbeitbarkeit, die inhaltliche Richtigkeit und der Nutzen für Umsetzung und Prüfung. Das folgt dem Qualitätsverständnis aus Kapitel 2 und der Unterscheidung von Eigenschaft, Bewertung und Wirkung bei Frattini et al. (2023, S. 507–520).

Im vorhandenen Repository lassen sich nur strukturelle Eigenschaften automatisch prüfen. Für eine inhaltliche Messung bräuchte man unabhängige Referenzinformationen oder fachkundige Urteile. Bewertet werden müsste nicht die einzelne Modellantwort ohne Kontext, sondern das Zusammenspiel aus Ausgangsticket, verfügbaren Belegen und erzeugtem Entwurf. Für jede neue Aussage wäre zu kennzeichnen, ob sie belegt, widersprüchlich, unentscheidbar oder frei ergänzt ist. Rückfragen wären nach Relevanz, Wichtigkeit und Beantwortbarkeit zu beurteilen. Mehrere Läufe derselben Konfiguration wären nötig, damit Zufallsschwankungen des Modells nicht mit Unterschieden zwischen Tickets verwechselt werden.

Auch praktischer Nutzen braucht Vergleichsdaten. Bearbeitungszeit, Zahl der Klärungsrunden und Korrekturaufwand wären gemeinsam mit Richtigkeit und Prüfbarkeit zu erheben. Eine schnelle Übernahme eines sachlich falschen Entwurfs wäre kein positiver Effekt. Da weder ein solches Ticketkorpus noch unabhängige Bewertungen vorliegen, lautet das Ergebnis für EF-4 *nicht untersucht*. Das bewertet nicht das Modell. Es markiert eine Grenze des verfügbaren Materials. Die in Kapitel 2 offene Frage, ob Kommentar- und Repository-Kontext bessere Vorschläge erzeugt als ein isolierter Issue-Titel, bleibt damit ebenfalls unbeantwortet.

6.5. Befunde entlang der Prozessgrenzen

6.5.1. Ereignisannahme und Zugriffsschutz

FA-1 fordert `@dev-assist` am Anfang der ersten inhaltlichen Zeile. Die Implementierung akzeptiert dagegen eine Erwähnung im Titel oder mitten im Text. FA-1 ist nicht erfüllt. Issue- und Kommentar-Ereignisse werden vereinheitlicht. Das interne Ereignis enthält jedoch keinen Autor. FA-2 ist deshalb nur teilweise erfüllt. Der Schlüssel gegen Doppelverarbeitung wird gesetzt, bevor geprüft wird, ob das Ereignis fachlich relevant ist. Er verwendet Projekt, Issue, Ereignisart sowie eine Lieferungs-ID oder `no-id`. Unterschiedliche legitime Lieferungen können innerhalb der kurzen Speicherzeit kollidieren. Neustarts und mehrere Instanzen werden nicht abgedeckt.

Die Webhook-Prüfung verwendet `x-gitlab-signature`, `x-gitlab-timestamp` und eine Prüfsumme über `timestamp:body`. Eine Zeitstempelprüfung fehlt. Ist kein Geheimnis konfiguriert, gilt ein Webhook selbst bei aktivierter Signaturpflicht als gültig. Nach den aktuellen Vorgaben von GitLab werden dagegen `webhook-id`, `webhook-timestamp` und `webhook-signature` verwendet; signiert wird `{message_id}.{timestamp}.{body}`, und der Zeitstempel muss aktuell sein (vgl. GitLab Documentation, o. J.a, Abschnitt "Signing tokens"). Die Tests bestätigen damit nur das implementierte Verfahren, nicht die Vorgaben von GitLab. NFA-6 ist nicht erfüllt.

Auch die manuellen Process- und Publish-Wege besitzen keinen eigenen Zugangsschutz. Projekt- und Issue-IDs werden nicht durchgängig als positive Ganzzahlen geprüft. Aus ihnen gebildete Speicherpfade besitzen deshalb keine hinreichend belegte Eingabegrenze. Diese Punkte betreffen den Zugang zum Kernprozess. Eine spätere Schema-Prüfung kann sie nicht ausgleichen.

6.5.2. Modellgrenze und Ausgabequalität

Die Anwendung akzeptiert nur ein JSON-Objekt mit den sechs Feldern Titel, Beschreibung, Akzeptanzkriterien, technischer Kontext, Lösungsvorschlag und offene Fragen. Falsche Typen und unbekannte Felder werden abgelehnt. Damit ist die strukturelle Antwortgrenze für die geprüften Fälle nachgewiesen. Das fachliche Zielmodell aus Kapitel 3 enthält darüber hinaus Quellenbasis, Risiken und Prüfschritte. Diese Kategorien

haben im kompakten Schema keine eigenen Felder.

Die OpenCode-Auswertung akzeptiert den ersten schemafähigen Kandidaten in Fundreihenfolge. Bekannte Beispiele im Prompt werden nicht ausdrücklich ausgeschlossen. Die Darstellung markiert leere Listen, lässt aber inhaltliche Platzhalter wie `unknown` unverändert. Eine formal gültige Ausgabe kann daher der falsche Kandidat oder fachlich unzureichend sein. Reale Modellqualität, Versionskompatibilität, wiederholte Läufe und die inhaltliche Verarbeitung von Anhängen wurden nicht untersucht.

6.5.3. Speicherung und Publish

Im Klärungsfall zeigt GitLab nur Rückfragen. Der Processor speichert dennoch den vollständigen Analysekontext. Der Publisher prüft weder, ob zuvor ein vollständiger Vorschlag sichtbar war, noch ob offene Fragen einer Freigabe entgegenstehen. Die Freigaberegeln, nur den vollständig sichtbaren und freigegebenen Stand zu veröffentlichen, ist damit nicht abgesichert.

Zusätzlich löscht der Publisher ausgewählte Dev-Assist-Kommentare vor der Issue-Aktualisierung. Schlägt diese Aktualisierung fehl, kann bereits Diskussionskontext verloren sein. FA-11 ist deshalb nicht erfüllt. Die in Kapitel 5 dargestellte Lücke ist keine Komfortfrage. Sie betrifft die Integrität des Freigabeprozesses.

6.6. Überleitung zu den Ergebnissen

EF-1 ist für den dokumentierten Lauf beantwortet. EF-2 bleibt auf ausgewählte Teilwege begrenzt. EF-3 weist Widersprüche zu Muss-Anforderungen und zu den Signaturvorgaben von GitLab nach. EF-4 ist nicht untersucht. Die verdichtete Statusübersicht folgt in Kapitel 7. Ursachen, Alternativerklärungen und Gültigkeit behandelt Kapitel 8.

7. Ergebnisse

Dieses Kapitel fasst die Einzelbefunde aus Kapitel 6 zusammen. Die Prüfwege und der Quellcode werden nicht erneut ausgebreitet. Beantwortet werden drei Fragen. Welche Anforderungen sind im untersuchten Stand nachweisbar erfüllt? Wo weicht der Ablauf vom Soll ab? Wie weit reicht der Nachweis? Ursachen und Folgen behandelt Kapitel 8. Die Forschungsfragen und die nächsten Schritte stehen in Kapitel 9.

7.1. Gesamtstand der Anforderungen

Tabelle 7.1 fasst die entscheidenden Ergebnisse zusammen. *Teilweise erfüllt* heißt: Ein Teil der Funktion ist vorhanden, der geforderte Umfang oder der Nachweis bleibt aber begrenzt. *Nicht erfüllt* heißt: Das beobachtete Verhalten widerspricht der Anforderung, oder eine notwendige Kontrolle fehlt. *Nicht untersucht* ist keine Fehlfunktion. Die Kategorie gilt, wenn zur fachlichen Wirkung keine geeigneten Daten vorliegen. Das ist bei der Modellantwort wichtig: Die formale Verarbeitung wurde geprüft, die fachliche Qualität nicht.

Tabelle 7.1: Konsolidierter Anforderungs- und Nachweisstand des untersuchten Prototyps (eigene Darstellung)

Anforderung oder Grenze	Ergebnis	Verdichtete Aussage
FA-1: Aktivierung	Nicht erfüllt	Eine Erwähnung im Titel oder mitten im Text startet den Dienst, obwohl nur der Anfang der ersten inhaltlichen Zeile erlaubt ist.
FA-2: Autorinformation	Teilweise erfüllt	Die interne Ereignisdarstellung enthält nicht, wer das Ticket oder den Kommentar geschrieben hat.
NFA-6: Authentifizierung	Nicht erfüllt	Die Prüfung ist optional, weicht von den Signaturvorgaben von GitLab ab und enthält keine Zeitstempelprüfung.
Manuelle Routen, Kennungen und Pfade	Nicht erfüllt	Zugriffsschutz und eine hinreichende Prüfung von IDs und daraus gebildeten Speicherpfaden fehlen.
Abgleich von Vorschau und gespeichertem Stand	Nicht erfüllt	Der gespeicherte Publish-Stand kann über den vollständig sichtbaren und bestätigten Vorschlag hinausgehen.
FA-11: Löschreihenfolge	Nicht erfüllt	Kommentare können gelöscht werden, bevor die Issue-Aktualisierung erfolgreich abgeschlossen ist.
NFA-7: Schutz vor Doppelverarbeitung	Teilweise erfüllt	Eine Filterung ist vorhanden. Sie gilt aber nur im laufenden Prozess und hängt von Feldern der GitLab-Lieferung ab, die nicht immer eindeutig sind.
FA-4 bis FA-8: Modellantwort	Form teilweise; Inhalt nicht untersucht	Schema und Verarbeitung sind für ausgewählte Fälle geprüft. Ob die Aussagen fachlich stimmen und nutzbar sind, wurde nicht untersucht.
Kontrollierter Gesamtablauf	Nicht nachgewiesen	Einzelne Schritte sind belegt. Offene Muss-Anforderungen sowie fehlende Gesamtläufe und Wirkungsdaten verhindern den Nachweis des gesamten Ablaufs.

Die Übersicht ist ungleichgewichtig. Die Kette aus Ereignisannahme, Kontext, geprüfter Analyse, Rückfrage oder Vorschlag und getrenntem Publish ist vorhanden. Die Abweichungen sitzen aber an den Stellen, die den Ablauf kontrollieren sollen: Aktivierung, Echtheit, manueller Zugriff und die Bindung zwischen sichtbarem Vorschlag und Veröf-

fentlichung. Bestandene Tests belegen die implementierte Verarbeitung. Sie belegen nicht automatisch die Sollanforderung. Ebenso belegen sie nicht, ob die Modellantworten fachlich stimmen.

7.2. Illustrierter Ablauf

Die Abweichungen sollen nicht nur als Tabelle erscheinen. Dieser Abschnitt verdichtet den Kernablauf an dem dokumentierten GitLab-Lauf aus Kapitel 5. Es handelt sich um ein eigens angelegtes Demonstrationsissue im untersuchten Projekt. Es sind weder betriebliche Originaldaten noch ein empirischer Evaluationsfall. Der Verlauf folgt dem Ausgabeschema und der Darstellungslogik des untersuchten Stands. Screenshots und der ausführlichere Ticketverlauf stehen in den Abbildungen 5.1 bis 5.3 sowie in Anhang A.

Ausgangslage ist ein unvollständiges Issue mit dem Titel “Screenshots in the ticket are ignored” und einer einzeiligen Beschreibung. Ziel, Umfang und Erfolgskriterium fehlen. Nach der Aktivierung durch `@dev-assist` lädt der Processor Issue und Kommentare. Er übergibt den begrenzten Kontext an den Analyseagenten und erhält ein JSON-Objekt mit den sechs erwarteten Feldern. Weil wesentliche Angaben fehlen, entstehen mindestens zwei inhaltlich relevante offene Fragen, hier zum Geltungsbereich der Bilder und zu ihrer Sichtbarkeit im Tickettext. GitLab zeigt in diesem Fall nur den Klärungskommentar. Lokal wird dennoch der vollständige Analysekontext gespeichert.

Beantwortet eine Person die Fragen im Issue, kann eine erneute Analyse Ziel, Akzeptanzkriterien und technischen Kontext in die vorgesehenen Felder überführen. Sichtbar wird dann ein strukturierter Vorschlag. Erst `@dev-assist publish` schreibt diesen Stand in Titel und Beschreibung. Im untersuchten Stand prüft der Publisher jedoch weder, ob zuvor ein vollständiger Vorschlag sichtbar war, noch ob offene Fragen einer Freigabe entgegenstehen. Das Beispiel macht denselben Befund wie Tabelle 7.1 konkret. Die Kette der Bausteine ist vorhanden. Die Freigaberegeln, nur den vollständig sichtbaren und freigegebenen Stand zu veröffentlichen, fehlt.

7.3. Zustände eines Vorschlags

Für die Einordnung der Ergebnisse ist der Zustand eines Vorschlags wichtiger als die bloße Existenz einer Kontextdatei. Ein kontrollierter Ablauf unterscheidet mindestens die Zustände *neu*, *in Analyse*, *in Klärung*, *als vollständige Vorschau sichtbar*, *ausdrücklich freigegeben* sowie *veröffentlicht* oder *Fehler*. Ein Analyseergebnis mit mehreren offenen Fragen darf demnach in die Klärung übergehen. Es darf noch nicht als veröffentlichungsfähiger Vorschlag gelten. Erst nachdem der vollständige Stand sichtbar gemacht und ausdrücklich freigegeben wurde, ist der Übergang zu Publish zulässig.

Die dafür erforderliche Freigaberegeln lautet:

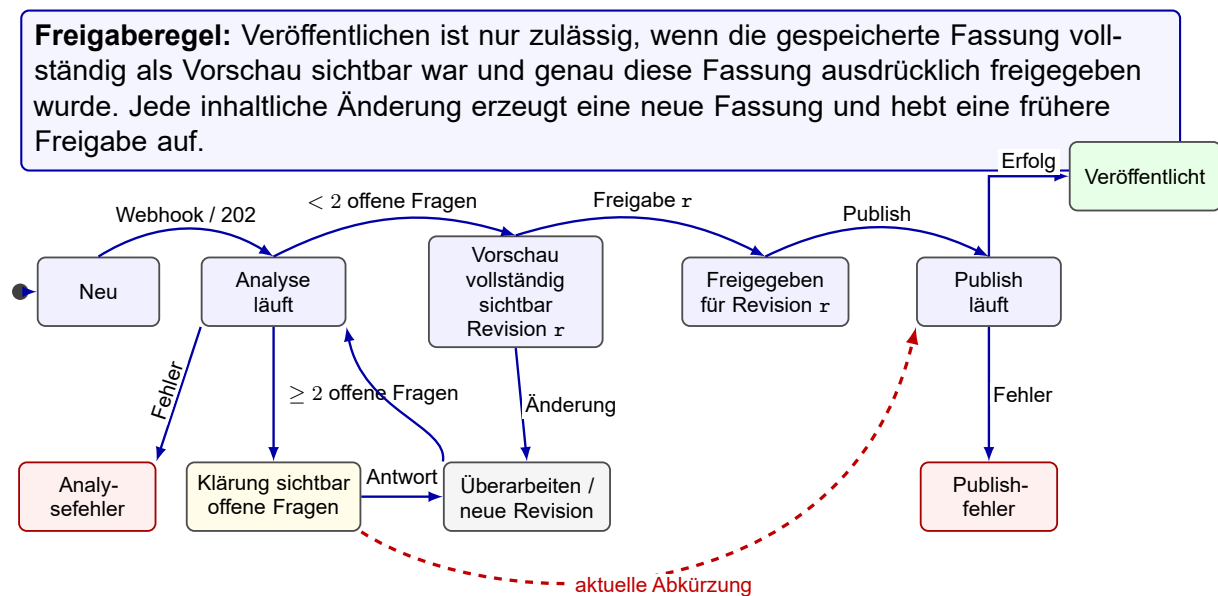
Freigaberegeln:

Die gespeicherte Fassung muss dieselbe sein, die vollständig sichtbar war,
und sie muss ausdrücklich freigegeben worden sein.

Abbildung 7.1 überführt die Regel in ein Zustandsmodell. Blaue Übergänge bilden den erforderlichen kontrollierten Ablauf. Der rot gestrichelte Pfad markiert die im untersuchten Stand mögliche Abkürzung.

Die Regel verbindet Sichtbarkeit, gleiche Fassung und menschliche Entscheidung. Erforderlich sind eine sichtbare Vorschau, eine Freigabe genau dieser Fassung und eine erneute Prüfung unmittelbar vor der Issue-Änderung. Der untersuchte Stand erzwingt das nicht. Während einer sichtbaren Klärung kann ein umfangreicherer Stand gespeichert und später ohne Statusprüfung übernommen werden. Die zeitliche Trennung von Analyse und Publish bildet daher noch keinen kontrollierten Prozess mit menschlicher Prüfung.

Auch Fehler müssen in diesem Modell einen sicheren Zustand ergeben. Scheitert die Issue-Aktualisierung, müssen Issue, Vorschau und Diskussion unverändert nachvollziehbar bleiben. Erst nach bestätigter Kernänderung darf das Löschen der Dev-Assist-Kommentare folgen. Die Prüfung der gezeigten Fassung und die fehlersichere Reihenfolge ergänzen einander.



Aktuelle Implementierungslücke: Die Anwendung speichert den vollständigen Analysekontext auch dann, wenn GitLab nur Rückfragen zeigt. Im gespeicherten Stand fehlen Status, Versionskennung und Freigabevermerk. Deshalb kann Publish den gespeicherten Vollvorschlag übernehmen, ohne zu prüfen, ob genau diese Fassung sichtbar und freigegeben war.

Abbildung 7.1.: Soll-Zustandsmodell eines Dev-Assist-Vorschlags und Abweichung der aktuellen Implementierung. Die Freigabe muss an dieselbe Fassung gebunden sein, die vollständig in GitLab sichtbar war. Der rot gestrichelte Übergang zeigt die fehlende Prüfung von Status, Fassung und Freigabe. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

7.4. Reichweite des Ergebnisses

Der Nachweis endet bei den untersuchten festen Programmteilen. Reale Gesamtläufe, fachliche Ticketqualität, Lastverhalten und Nutzung im Team wurden nicht geprüft. Zeitersparnis oder Fehlerreduktion lassen sich daher nicht ableiten. Der Prototyp ist ein technischer Machbarkeitsnachweis ausgewählter Bausteine. Er ist kein Nachweis eines kontrollierten oder produktionsreifen Gesamtablaufs.

8. Diskussion

Kapitel 6 dokumentiert die Einzelbelege. Tabelle 7.1 hält den verdichteten Ergebnisstand fest. Die Diskussion wiederholt diese Befunde nicht. Sie ordnet ein, was sie bedeuten, prüft andere Erklärungen und begrenzt, wie weit die Aussagen gelten. Die Forschungsfragen werden erst in Kapitel 9 abschließend beantwortet.

8.1. Interpretation des Ergebnisbeitrags

Dev-Assist setzt an einem bekannten Informationsproblem an. In technischen Tickets fehlen häufig Reproduktionsschritte, Systemausgaben, Testfälle oder entscheidungsrelevanter Kontext. Wesentliche Angaben entstehen oft erst im Dialog (vgl. Bettenburg et al., 2008, S. 308 f.; Breu et al., 2010, S. 301 ff.; Chaparro et al., 2017, S. 396 f.). Der nachgewiesene Beitrag liegt in der Anwendung um das Modell herum: Kontext wird gebündelt, die Ausgabe folgt einem festen Schema, die Issue-Änderung bleibt getrennt. Tabelle 7.1 begrenzt die Reichweite. Ohne gesicherte Herkunft der Ereignisse, ohne Bindung der Freigabe an den gezeigten Stand und ohne sichere Fehlerreihenfolge bleibt es ein Machbarkeitsnachweis ausgewählter Bausteine, kein kontrollierter Gesamtprozess.

8.1.1. Formale Struktur und fachliche Qualität

Das Ausgabeschema verbessert zunächst die maschinelle und visuelle Struktur. Ungültige Typen oder unbekannte Felder können zurückgewiesen werden. Nach Lucassen et al. sind Form, Bedeutung und praktische Nutzbarkeit jedoch zu trennen (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Ein wohlgeformtes Akzeptanzkriterium kann fachlich falsch sein. Ein korrektes Detail kann irrelevant sein. Ein vollständiger Abschnitt kann nur scheinbar vollständig sein.

Aktuelle empirische Forschung bestätigt diese Grenze. Bei Lubos et al. war die anfängliche Übereinstimmung zwischen menschlicher Bewertung und Modellbewertung teilweise schwach, obwohl Erklärungen und Überarbeitungen häufig als hilfreich galten (vgl. Lubos et al., 2024, S. 389–397). Paiva et al. berichten hohe sprachliche Qualitätsbewer-

tungen, aber begrenzte exakte Übereinstimmung zwischen menschlichen Urteilen und einer automatischen Modellbewertung sowie teilweise spekulative Ergänzungen (vgl. Paiva et al., 2026, Abschnitte 3–5). Mahbub et al. zeigen zusätzlich begrenzte Treffsicherheit bei Mehrdeutigkeit, Widersprüchen und Unvollständigkeit, insbesondere wenn Bezüge über das einzelne Dokument hinaus und Fachwissen fehlen (vgl. Mahbub et al., 2024, S. 171979–171989). Dev-Assist braucht deshalb die Schemaerfüllung als notwendige Verarbeitungsbedingung. Sie ersetzt kein fachliches Qualitätsurteil.

8.1.2. Rückfragen und Unsicherheit

Der Rückfragemodus macht Unsicherheit zum Gegenstand gemeinsamer Klärung. K C et al. zeigen an realen Feature Requests, dass aus Mehrdeutigkeit und Unvollständigkeit konkrete Rückfragen abgeleitet werden können; ihre Beantwortung und Einordnung verbleiben jedoch bei Menschen (vgl. K C et al., 2025, Abschnitte III–V). Die Anzahl offener Fragen ist dabei kein ausreichendes Maß für ihre Bedeutung. Eine einzelne Sicherheitsentscheidung kann den Fortgang blockieren. Mehrere Detailfragen können nachgeordnet sein. Die Schwelle des Prototyps – mindestens zwei inhaltlich relevante Fragen – ist daher nur eine grobe Regel.

Die Gestaltung braucht deshalb Rückfragen, die die zugrunde liegende Informationslücke und deren Auswirkung auf die Entscheidung erkennen lassen. Eine Antwort darf erst dann als geklärt gelten, wenn ihre Herkunft nachvollziehbar ist. Andernfalls bleibt die Unsicherheit im Entwurf sichtbar. Menschliche Beteiligung bezeichnet hier fachliche Klärung. Sie ist nicht bereits die spätere formale Freigabe.

8.2. Nutzenhypothesen, Aufwand und Alternativen

Plausibel ist eine Entlastung bei wiederkehrender Zusammenführung. Angaben werden geordnet, beantwortete Fragen zusammengeführt und im bestehenden GitLab-Arbeitsort in eine einheitliche Struktur überführt. Gemessen wurden weder Bearbeitungszeit noch Klärungsrunden, Übernahmequote, Korrekturaufwand oder Fehlerquote. Aus dem vorhandenen Material folgt keine Wirkungsbehauptung.

Tabelle 8.1 ordnet den Aufwand der Alternativen ein, ohne einzelne Modellkosten auszu-

weisen. Prüfzeit, Fehlerrate und Ticketmenge sind nur gemeinsam aussagekräftig. Eine beobachtete Verbesserung könnte bereits durch ein verpflichtendes Formular, einfachere Ausgangstickets oder den bloßen Neuheitseffekt entstehen. Saleem et al. berichten bei mehreren Aufgaben des Requirements Engineering bessere Ergebnisse spezialisierter Verfahren; generative Modelle waren insbesondere bei der Beantwortung von Fragen konkurrenzfähig (vgl. Saleem et al., 2025, Abschnitte “Results” und “Conclusion”). Relevante Vergleichsgrundlagen sind daher manuelle Aufbereitung, feste Vorlagen und spezialisierte Modelle.

Tabelle 8.1: Wirtschaftliche Einordnung der Aufbereitungsalternativen ohne Wirkungsnachweis

Alternative	Aufwand und Nutzen	Grenze
Issue-Vorlage	Keine Modellkosten; einheitliche Felder ohne zusätzliche Oberfläche.	Keine dialogische Zusammenführung über Kommentare.
Manuelle Aufbereitung	Volle fachliche Kontrolle; Aufwand steigt mit verteiltem Kontext.	Wiederholte Rückfragen und uneinheitliche Struktur.
Dev-Assist (PoC)	Keine zusätzliche Oberfläche; Struktur und Rückfragen im Issue. Variabel: Modellaufruf, menschliche Prüfung, Webhook-Betrieb.	Fachliche Güte ungemessen; Sicherheits- und Integritätslücken offen.

Mehr Kontext verbessert Ergebnisse nicht automatisch. ReqRAG erreichte in der industriellen Bewertung von Ibtasham et al. für rund 70 Prozent der Antworten positive Nützlichkeits- und Relevanzurteile, aber keine vollständige Übereinstimmung (vgl. Ibtasham et al., 2025, S. 277–292). Veraltete oder unpassende Fundstellen können eine Antwort verschlechtern. Eine Wirkungsevaluation müsste dieselben Tickets unterschiedlicher Art unter mehreren Bedingungen bearbeiten. Die Varianten wären zu beurteilen, ohne dass erkennbar ist, welche vom Assistenten stammt. Ob Aussagen belegt sind und wie hoch der Aufwand ist, wäre zu messen. Qualitätssteigerung und Zeitgewinn bleiben offene Hypothesen.

8.3. Zielkonflikte der Architektur

Das feste Ausgabeschema macht die Ausgabe beherrschbar und begrenzt zugleich, was sich ausdrücken lässt. Wenige typisierte Felder erleichtern das Einlesen und die Darstellung. Quellenbasis, Risiken und Annahmen bleiben jedoch in allgemeinen Textlisten verborgen. Zusätzliche Felder erhöhen wiederum die Abhängigkeit zwischen Prompt, Typen, Prüfung, Darstellung und Publish. Das kompakte Ausgabeschema ist deshalb eine begrenzte technische Form. Es ist kein vollständiges Modell aller Merkmale von Ticketqualität.

Die Anwendung bestätigt die Annahme einer Lieferung sofort (HTTP 202), bevor die Analyse fertig ist. Annahme der Nachricht und fachlicher Abschluss fallen dadurch auseinander. Ohne dauerhaft gespeicherten Auftragszustand sind Fehler und noch laufende Verarbeitung schwer unterscheidbar. Ähnlich stehen Übersichtlichkeit und Nachvollziehbarkeit in Spannung. Das Löschen von Dev-Assist-Kommentaren kann ein Issue bereinigen, zugleich aber Entscheidungs- und Fehlerhistorie entfernen. Lokale Kontextdateien erhöhen die Reproduzierbarkeit, erzeugen aber zusätzliche Datenkopien. Diese Konflikte erfordern klare Zustände, Herkunftsangaben, Berechtigungen und Aufbewahrungsregeln. Ein besseres Modell allein löst sie nicht.

8.4. Organisatorische Einbettung menschlicher Kontrolle

Menschliche Kontrolle im Prozess ist nicht bereits durch ein Publish-Kommando verwirklicht. Kontrolle besitzt nur dann fachliche Bedeutung, wenn eine zuständige Person den vollständigen Vorschlag samt Quellen prüft. Flüssige, wiederholt plausible Ausgaben können sonst eine nur formale Bestätigung begünstigen. Die Bindung der Freigabe an die gezeigte Fassung aus Kapitel 7 ist daher notwendig, aber nicht hinreichend.

Im Prozess sind fachliche Eingabe, assistierte Strukturierung und Verantwortung für den verbindlichen Stand zu unterscheiden. Besonders bei Sicherheits-, Datenschutz- oder Architekturannahmen muss erkennbar sein, wer welche Fassung bestätigt hat. Autor, Zeitpunkt und Versionskennung sind damit Bestandteile der Verantwortungszuordnung. Zugleich sollte die Oberfläche direkte Belege, Ableitungen aus dem Repository und offene Modellannahmen unterscheidbar darstellen. Die begrenzte Übereinstimmung menschlicher und automatischer Qualitätsurteile spricht gegen einen bloß

allgemeinen Warnhinweis (vgl. Lubos et al., 2024, S. 389–397; Paiva et al., 2026, Abschnitte 3–5).

Prompt, Schema, Modell und Kontextauswahl verändern das System auch ohne sichtbare Oberflächenänderung. Sie brauchen Versionierung, dokumentierte Freigabe und Wiederholungsprüfungen. Abweichungen müssen einer Konfiguration zugeordnet werden können. Technischer Nachweis und organisatorische Verantwortung müssen dabei denselben Stand beschreiben.

8.5. Gültigkeit der Untersuchung

Tabelle 8.2 fasst die wesentlichen Bedrohungen zusammen. Sie bewertet nicht das System, sondern begrenzt, wie die Belege gelesen werden dürfen.

Tabelle 8.2: Zentrale Bedrohungen der Gültigkeit und ihre Konsequenzen

Frage	Bedrohung	Folge für die Interpretation
Wird das Richtige gemessen?	Build, Testzahl und Schemaerfüllung messen technische Teilfunktionen, nicht Ticketqualität oder Prozessnutzen.	Kein Nachweis fachlicher Qualität, Testabdeckung oder Produktivität
Folgt die Schlussfolgerung aus den Daten?	Test und Implementierung können dieselbe falsche Annahme über die GitLab-Dokumentation enthalten.	Interne Stimmigkeit bestätigt keine Übereinstimmung mit der GitLab-Dokumentation
Gilt das auch außerhalb dieses Falls?	Einzelner Repository-Stand; keine realen Teams, Ticketmengen, Last- oder Modellvarianten	Keine Erfolgsquoten und keine statistische Verallgemeinerung auf andere Projekte
Wäre das Ergebnis bei Wiederholung dasselbe?	Dokumentierter technischer Lauf, aber keine wiederholte Bewertung zufälliger Modellantworten	Technischer Stand reproduzierbar; Streuung realer Modellantworten unbekannt

Die stärkste Fehldeutung wäre die Gleichsetzung bestandener Softwaretests mit Ticketqualität. Die Kategorie *nicht untersucht* verhindert, fehlende Wirkungsbelege umzudeuten. Ausführung, Quellcode und GitLab-Dokumentation wurden gemeinsam herangezogen. Das ersetzt keine unabhängige Zweitprüfung des gesamten Abgleichs. Ticketart, Fachgebiet, Sprache sowie Modell- und GitLab-Version können Ergebnisse verändern. Die dokumentierte Umgebung erhöht die Reproduzierbarkeit der technischen Bestandsaufnahme. Sie erhöht nicht die Stabilität der Modellantworten.

8.6. Risiken und verantwortbarer Einsatz

Eine formale Prüfung verhindert weder erfundene Aussagen noch kritische Auslassungen. NIST fasst überzeugend präsentierte Falschaussagen als Halluzination, auch Confabulation genannt (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Quellenhinweise je Aussage, sichtbare offene Punkte und ein Abgleich zwischen Eingangskontext und Vorschlag sind deshalb aussagekräftiger als eine einzige Vollständigkeitskennzahl.

Tickettexte, Kommentare und Repository-Dateien sind nicht vertrauenswürdige Daten. Eingebettete Anweisungen im Tickettext (Prompt Injection) können die Analyse verzerren, obwohl das Modell keine direkten GitLab-Rechte besitzt (vgl. Liu et al., 2023, S. 1; OWASP, 2025a, Abschnitt “LLM01:2025 Prompt Injection”). Der getrennte Publish-Schritt begrenzt übermäßige Handlungsfähigkeit. Er schützt aber nur, wenn genau die unveränderte, gezeigte Fassung freigegeben wird (vgl. OWASP, 2025b, Abschnitt “LLM06:2025 Excessive Agency”). Schema und getrennte Berechtigungen verringern somit die Schadenswirkung. Sie ersetzen nicht die Notwendigkeit menschlicher Inhaltsprüfung.

Issues, Logs und Quellcode können personenbezogene Angaben, interne URLs, Schwachstellen oder Zugangsgeheimnisse enthalten. Zusätzlicher Kontext erhöht Datenmenge und Angriffsfläche. Vor einer Übertragung an ein externes Modell sind Datenminimierung, Erkennung geheimer Zugangsdaten und Zweckbindung erforderlich. Lokale Kontextdateien brauchen Zugriffsrechte und Löschfristen. Die Oberfläche sollte außerdem zwischen belegten, abgeleiteten und offenen Angaben unterscheiden. Flüssige Sprache darf keine unangemessene Sicherheit vermitteln.

8.7. Übertragbarkeit

Webhook-Formate, Schnittstellen und die Bedeutung von Publish sind GitLab-spezifisch. Übertragbar ist das Muster aus Ereignisfilter, begrenztem Kontext, geprüfter Modellantwort, sichtbarer Vorschau und ausdrücklicher Freigabe. In einem anderen Issue-System müssen Echtheitsprüfung, Rollen, Schutz vor Doppelverarbeitung und Ausgabeschema neu geprüft werden. Eine Übernahme der Bausteine überträgt keine nachgewiesene Prozesseigenschaft.

Auch fachlich bleibt das Ausgabeschema eine Mindeststruktur. Fehlerberichte, Feature-Tickets sowie Wartungs- und Sicherheitsaufgaben brauchen unterschiedliche Zusatzinformationen. Sprache, Fachgebiet, Modellversion, Prompt und Kontextabruf verändern zudem die Modellleistung. Übertragbar ist daher das Gestaltungsprinzip fester Vertrauensgrenzen und menschlicher Verantwortung. Eine Erfolgsquote wurde in dieser Arbeit nicht erhoben und ist nicht übertragbar.

8.8. Diskussionsfazit

Die Diskussion ordnet Dev-Assist als technische Assistenz in einem menschlichen Arbeitsablauf ein. Formale Verarbeitbarkeit, fachliche Bewertung und organisatorische Verantwortung brauchen jeweils eigene Kontrollen und Belege. Kapitel 9 beantwortet auf dieser Grundlage die Forschungsfragen und priorisiert den weiteren Bedarf.

9. Fazit und Ausblick

Die Arbeit untersuchte den Informationsbedarf umsetzbarer GitLab-Tickets und die kontrollierte Einbindung eines KI-Assistenten. Dazu entstanden ein fachliches Zielbild und der TypeScript-/Express-Prototyp Dev-Assist. Ergänzend erfolgte ein systematischer Abgleich mit Anforderungen, GitLab-Dokumentation und Tests. Abschließend werden beide Forschungsfragen beantwortet und die nächsten Schritte priorisiert.

9.1. Beantwortung der Forschungsfragen

9.1.1. Erforderliche Informationen eines Tickets

Die erste Forschungsfrage lautet: *Welche Informationen muss ein GitLab-Ticket enthalten, damit Entwicklerinnen und Entwickler es nachvollziehbar umsetzen und überprüfen können?* Die Untersuchung ergibt sechs miteinander verbundene Informationsarten:

1. **Gegenstand und Ziel:** Titel und Zielbeschreibung benennen Änderung oder Störung und den angestrebten Nutzen.
2. **Kontext und Abgrenzung:** Rollen, Systeme, Voraussetzungen, Randbedingungen und Nicht-Ziele grenzen den Auftrag ein.
3. **Erwartetes Verhalten:** Soll- und Ist-Zustand müssen unterscheidbar sein. Fehlerberichte brauchen insbesondere Reproduktionsschritte.
4. **Prüfkriterien:** Beobachtbare Akzeptanzkriterien machen das Ergebnis überprüfbar.
5. **Technische Belege:** Logs, Fehlermeldungen und Repository-Hinweise ergänzen die Beschreibung mit nachvollziehbarer Herkunft.
6. **Sichtbare Unsicherheit:** Offene Entscheidungen, Widersprüche und fehlende Angaben bleiben erkennbar und führen zu konkreten Rückfragen.

Die Kategorien bilden einen ticketartübergreifenden Mindestbestand. Sie sind kein star-

res Formular. Entscheidend sind Relevanz, Nachvollziehbarkeit und Prüfbarkeit, nicht Textmenge. Das kompakte Ausgabeschema des Prototyps bildet den Kern ab. Sein Lösungsvorschlag bleibt ein veränderbarer Entwurf und keine Quelle fachlicher Wahrheit (vgl. Bettenburg et al., 2008, S. 308 f.; Lucassen et al., 2016, S. 386 f.; K C et al., 2025, S. 104–116).

9.1.2. Kontrollierte Integration eines KI-Assistenten

Die zweite Forschungsfrage lautet: *Wie kann ein KI-gestützter Assistent technisch so in einen GitLab-basierten Entwicklungsworkflow integriert werden, dass Analyse, Rückfragen, Vorschläge und Freigabe kontrolliert ablaufen?* Aus Architektur, Implementierung und Evaluation ergeben sich sechs notwendige Kontrollen:

1. **Ereignisannahme:** Echtheitsprüfung, Zeitstempel-, Typ- und ID-Prüfung, Filter für Bot-Nachrichten, Aktivierungsregeln und Schutz vor Doppelverarbeitung begrenzen zulässige Ereignisse.
2. **Kontextaufbereitung:** Die Anwendung wählt und begrenzt Issue-, Kommentar- und Repository-Daten und behandelt sie als nicht vertrauenswürdig.
3. **Trennung vom Modell:** Das Modell erhält nur den vorbereiteten Kontext und keine produktiven GitLab-Rechte.
4. **Antwortgrenze:** Einlesen, festes Schema, Herkunftsregeln und Auswahl der verwendeten Antwort begrenzen die Modellantwort nach festen Regeln.
5. **Sichtbarer Zwischenstand:** Nur die vollständig sichtbare und danach freigegebene Fassung ist veröffentlichtbar.
6. **Fehlersichere Änderung:** Ein autorisierter Publish-Schritt ändert zuerst das Issue. Nachgeordnetes Löschen von Dev-Assist-Kommentaren darf den Ausgangsstand bei Fehlern nicht zerstören.

Kontrolle entsteht durch die Anwendung mit festen Regeln, nicht durch den Prompt allein. Der Prototyp zeigt wesentliche Bausteine. Tabelle 7.1 weist jedoch offene Muss-Anforderungen an Sicherheits- und Integritätsgrenzen aus. Das Architekturprinzip ist

damit als Zielmodell tragfähig. Der kontrollierte Gesamtablauf ist im untersuchten Stand aber nicht nachgewiesen. Fachliche Informationsqualität und technische Prozessintegrität sind gemeinsam erforderlich.

9.2. Wissenschaftlicher und praktischer Beitrag

Die Arbeit leistet drei Beiträge: erstens ein aus der Literatur abgeleitetes Qualitätsmodell, zweitens ein Architekturmodell mit überprüfbaren Übergängen zwischen Ereignis, Modellantwort, Vorschau und Issue-Änderung und drittens eine ausführbare Referenzimplementierung mit dokumentiertem Nachweisstand. Formale Struktur, Faktentreue und praktische Nutzbarkeit werden dabei getrennt. Die Freigabe genau der gezeigten Fassung macht eine häufig übersehene Integritätsanforderung sichtbar.

Die Aussagekraft bleibt auf das untersuchte Projekt begrenzt. Die Beiträge liegen in der Übersetzung der Literatur in prüfbare Ticketfelder, in der Architektur und in einer prüfbaren Bestandsaufnahme. Messwerte für Zeitgewinn oder Qualitätssteigerung sind nicht Gegenstand des Nachweises.

9.3. Priorisierter Ausblick

Die weitere Arbeit muss sachlichen Abhängigkeiten folgen. Zuerst ist die Übereinstimmung mit der GitLab-Dokumentation herzustellen. Danach kann fachliche Qualität belastbar untersucht werden. Erst auf dieser Grundlage lässt sich praktischer Prozessnutzen bewerten. Tabelle 9.1 verdichtet diese Reihenfolge zu drei Entwicklungs- und Evaluationsstufen.

Tabelle 9.1: Priorisierte Roadmap vom Machbarkeitsnachweis zur belastbaren Evaluation

Stufe	Ziel und Maßnahmen	Eintritts- oder Erfolgskriterium
1: Schnittstellen- und Prozesssicherheit	Signaturvorgaben von GitLab, manuelle Zugänge, IDs, Pfade, Freigabe der gezeigten Fassung, Publish-Reihenfolge und Schutz vor Doppelverarbeitung absichern	Positive und negative Integrationstests belegen sichere Eingangs-, Freigabe- und Fehlerpfade
2: Fachliche Bewertung und feste Regeln	Sammlung von Tickets unterschiedlicher Art; Vergleich mit manueller und regelbasierter Aufbereitung; wiederholte Läufe und unabhängige Bewertungen	Ob Aussagen belegt sind, ob der Kontext ausreicht, ob Rückfragen treffen und wie hoch der Korrekturaufwand ist, ist vorab messbar festgelegt
3: Betrieb und Erweiterung	Auftragswarteschlange, Überwachung und Betrieb mit mehreren Instanzen; danach Anhänge, Profile und weitere Ereignisse	Der Mindeststandard bleibt unter Last und Teilausfällen erhalten; Erweiterungen besitzen eigene Sicherheits- und Wiederholungsprüfungen

Die Reihenfolge lautet: Übereinstimmung mit der GitLab-Dokumentation vor fachlicher Qualität und diese vor praktischem Prozessnutzen. Andernfalls lassen sich Modell- und Prozessfehler nicht trennen. Erweiterungen müssen dieselben Herkunfts-, Berechtigungs- und Freigabegrenzen wie der Kernprozess einhalten.

9.4. Schlussfolgerung

Die zentrale Schlussfolgerung lautet: Generative Modelle können Ticketinhalte strukturieren. Verantwortlichkeit und Änderungsrecht verbleiben bei der Anwendung, die nach festen Regeln arbeitet und nur die gezeigte Fassung freigibt. Die Roadmap übersetzt diese Trennung in überprüfbare nächste Schritte: zuerst Schnittstellen und Zustände, dann fachliche Güte und anschließend betriebliche Wirkung.

Quellen

Bashir, Sarmad; Ferrari, Alessio; Khan, Abbas; Strandberg, Per Erik; Haider, Zulqarnain; Saadatmand, Mehrdad; Bohlin, Markus 2025. „Requirements Ambiguity Detection and Explanation with LLMs: An Industrial Study“, in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, S. 620–631. IEEE. <https://doi.org/10.1109/ICSME64153.2025.00063>.

Bass, Len; Clements, Paul; Kazman, Rick 2021. *Software Architecture in Practice*. 4. Aufl. Boston: Addison-Wesley.

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, S. 308–318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Bo, Lili; Ji, Wangjie; Sun, Xiaobing; Zhang, Ting; Wu, Xiaoxue; Wei, Ying 2024. „ChatBR: Automated Assessment and Improvement of Bug Report Quality Using ChatGPT“, in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, S. 1472–1483. New York: ACM. <https://doi.org/10.1145/3691620.3695518>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports: Improving Cooperation Between Developers and Users“, in *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work*, S. 301–310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Brown, Tom B.; Mann, Benjamin; Ryder, Nick; Subbiah, Melanie; Kaplan, Jared; Dhariwal, Prafulla et al. 2020. „Language Models Are Few-Shot Learners“, in *Advances in Neural Information Processing Systems* 33, S. 1877–1901. <https://arxiv.org/abs/2005.14165>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, S. 396–407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Easterbrook, Steve; Singer, Janice; Storey, Margaret-Anne; Damian, Daniela 2008. „Selecting Empirical Methods for Software Engineering Research“, in Shull, Forrest;

Singer, Janice; Sjøberg, Dag I. K. (Hrsg.): *Guide to Advanced Empirical Software Engineering*, S. 285–311. London: Springer. https://doi.org/10.1007/978-1-84800-044-5_11.

Express Documentation o. J. *Express: Node.js Web Application Framework*. <https://expressjs.com/> (Zugriff vom 31.07.2026).

Frattoni, Julian; Montgomery, Lloyd; Fischbach, Jannik; Mendez, Daniel; Fucci, Davide; Unterkalmsteiner, Michael 2023. „Requirements Quality Research: A Harmonized Theory, Evaluation, and Roadmap“, in *Requirements Engineering* 28, 4, S. 507–520. <https://doi.org/10.1007/s00766-023-00405-y>.

GitLab Documentation o. J.a. *Webhooks*. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 31.07.2026).

GitLab Documentation o. J.b. *Webhook Events*. https://docs.gitlab.com/user/project/integrations/webhook_events/ (Zugriff vom 31.07.2026).

GitLab Documentation o. J.c. *Issues API*. <https://docs.gitlab.com/api/issues/> (Zugriff vom 31.07.2026).

GitLab Documentation o. J.d. *Notes API*. <https://docs.gitlab.com/api/notes/> (Zugriff vom 31.07.2026).

Hohpe, Gregor; Woolf, Bobby 2003. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston: Addison-Wesley. <https://www.enterpriseintegrationpatterns.com/>.

Huang, Lei et al. 2023. „A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions“. arXiv:2311.05232. <https://doi.org/10.48550/arXiv.2311.05232>.

Ibtasham, Md Saleh; Bashir, Sarmad; Abbas, Muhammad; Haider, Zulqarnain; Saadatmand, Mehrdad; Cicchetti, Antonio 2025. „ReqRAG: Enhancing Software Release Management through Retrieval-Augmented LLMs: An Industrial Study“, in Hess, Anne; Susi, Angelo (Hrsg.): *Requirements Engineering: Foundation for Software Quality. 31st International Working Conference, REFSQ 2025, Lecture Notes in Computer Science 15588*, S. 277–292. Cham: Springer. https://doi.org/10.1007/978-3-031-88531-0_20.

Inayat, Irum; Salim, Siti Salwah; Marczak, Sabrina; Daneva, Maya; Shamshirband,

Shahaboddin 2015. „A Systematic Literature Review on Agile Requirements Engineering Practices and Challenges“, in *Computers in Human Behavior* 51, S. 915–929. <https://doi.org/10.1016/j.chb.2014.10.046>.

ISO/IEC/IEEE 2018. *ISO/IEC/IEEE 29148:2018. Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*. Öffentliche Katalogseite. <https://www.iso.org/standard/72089.html> (Zugriff vom 31.07.2026).

K C, Pragyan; Ghandiparsi, Rambod; Herron, Thomas; Heaps, John; Hosseini, Mitra Bokaei 2025. „Demystifying Feature Requests: Leveraging LLMs to Refine Feature Requests in Open-Source Software“, in *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, S. 104–116. IEEE. <https://doi.org/10.1109/RE63999.2025.00020>.

Krishna, Madhava; Gaur, Bhagesh; Verma, Arsh; Jalote, Pankaj 2024. „Using LLMs in Software Requirements Specifications: An Empirical Evaluation“, in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, S. 475–483. IEEE. <https://doi.org/10.1109/RE59067.2024.00056>.

Liu, Yi et al. 2023. „Prompt Injection Attack against LLM-Integrated Applications“. arXiv:2306.05499. <https://doi.org/10.48550/arXiv.2306.05499>.

Lubos, Sebastian; Felfernig, Alexander; Tran, Thi Ngoc Trang; Garber, Damian; El Mansi, Merfat; Erdeniz, Seda Polat; Le, Viet-Man 2024. „Leveraging LLMs for the Quality Assurance of Software Requirements“, in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, S. 389–397. IEEE. <https://doi.org/10.1109/RE59067.2024.00046>.

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving Agile Requirements: The Quality User Story Framework and Tool“, in *Requirements Engineering* 21, 3, S. 383–403. <https://doi.org/10.1007/s00766-016-0250-x>.

Mahbub, Taslim; Dghaym, Dana; Shankarnarayanan, Aadhith; Syed, Taufiq; Shapsough, Salsabeel; Zualkernan, Imran 2024. „Can GPT-4 Aid in Detecting Ambiguities, Inconsistencies, and Incompleteness in Requirements Analysis? A Comprehensive Case Study“, in *IEEE Access* 12, S. 171972–171992. <https://doi.org/10.1109/ACCESS.2024.3464242>.

National Institute of Standards and Technology 2024. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.600-1>.

Node.js Documentation o. J.a. *Node.js*. <https://nodejs.org/en> (Zugriff vom 31.07.2026).

Node.js Documentation o. J.b. *HTTP*. <https://nodejs.org/api/http.html> (Zugriff vom 31.07.2026).

OpenCode Documentation o. J.a. *Agents*. <https://opencode.ai/docs/agents/> (Zugriff vom 31.07.2026).

OpenCode Documentation o. J.b. *CLI*. <https://opencode.ai/docs/cli/> (Zugriff vom 31.07.2026).

OWASP 2025a. *LLM01:2025 Prompt Injection*. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Zugriff vom 31.07.2026).

OWASP 2025b. *LLM06:2025 Excessive Agency*. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/> (Zugriff vom 31.07.2026).

Paiva, Guilherme; Canedo, Edna Dias; Rocha Filho, Geraldo Pereira 2026. „From Issue Titles to Requirements: An Empirical Study of Large Language Models and Prompt Engineering Strategies“, in *Requirements Engineering* 31, Artikel 7. <https://doi.org/10.1007/s00766-026-00462-z>.

Saleem, Summra; Asim, Muhammad Nabeel; Van Elst, Ludger; Dengel, Andreas 2025. „Generative Language Models Potential for Requirement Engineering Applications: Insights into Current Strengths and Limitations“, in *Complex & Intelligent Systems* 11, 6, Artikel 278. <https://doi.org/10.1007/s40747-024-01707-6>.

Schick, Timo et al. 2023. „Toolformer: Language Models Can Teach Themselves to Use Tools“, in *Advances in Neural Information Processing Systems* 36, S. 68539–68551. <https://arxiv.org/abs/2302.04761>.

Song, Yang; Chaparro, Oscar 2020. „BEE: A Tool for Structuring and Analyzing Bug

Reports“, in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, S. 1551–1555. New York: ACM. <https://doi.org/10.1145/3368089.3417928>.

Tizard, James; Devine, Peter; Wang, Hechen; Blincoe, Kelly 2023. „A Software Requirements Ecosystem: Linking Forum, Issue Tracker, and FAQs for Requirements Management“, in *IEEE Transactions on Software Engineering* 49, 4, S. 2381–2393. <https://doi.org/10.1109/TSE.2022.3219458>.

TypeScript Documentation o. J. *The TypeScript Handbook*. <https://www.typescriptlang.org/docs/handbook/intro.html> (Zugriff vom 31.07.2026).

Werner, Nico 2026. *Dev-Assist*. Interner Projektstand, Git-Commit d179eb22 (vollständiger Hash in Kapitel 6) vom 26.07.2026, geprüft am 31.07.2026; Artefaktübersicht siehe Tabelle 6.2.

Xi, Zhiheng et al. 2023. „The Rise and Potential of Large Language Model Based Agents: A Survey“. arXiv:2309.07864. <https://doi.org/10.48550/arXiv.2309.07864>.

Yao, Shunyu et al. 2023. „ReAct: Synergizing Reasoning and Acting in Language Models“, in *International Conference on Learning Representations 2023*. https://openreview.net/forum?id=WE_vluYUL-X.

Zhao, Wayne Xin et al. 2023. „A Survey of Large Language Models“. arXiv:2303.18223. <https://doi.org/10.48550/arXiv.2303.18223>.

A. Illustrierter Ticketverlauf

Der folgende Verlauf dokumentiert denselben GitLab-Lauf, den die Abbildungen 5.1 bis 5.3 zeigen. Das Issue wurde im untersuchten Dev-Assist-Projekt angelegt. Es verwendet das Ausgabeschema und die Formatter-Logik des Prototyps. Betriebliche Originaldaten werden nicht verwendet. Der Verlauf belegt den sichtbaren Prozess, nicht die semantische Qualität beliebiger Modellantworten.

Ausgangsticket

Titel: Screenshots in the ticket are ignored

Beschreibung:

@dev-assist

If an issue has a screenshot attached, Dev-Assist somehow does not use it. That should work better.

Fehlend sind Ziel, Umfang, sichtbares Erfolgskriterium und die Abgrenzung. Nach FA-6 müssen daraus konkrete Rückfragen entstehen.

Sichtbarer Klärungskommentar

Im untersuchten Stand erzeugt der Processor bei mindestens zwei substanziellen offenen Fragen einen Klärungskommentar. Im dokumentierten Lauf lauteten die Fragen:

- Soll die Screenshot-Unterstützung nur Anhänge am Issue betreffen oder auch Bilder in späteren Kommentaren?
- Sollen Inhalte aus Screenshots im erzeugten Tickettext sichtbar werden oder nur intern in die Analyse einfließen?

Lokal wird in diesem Fall dennoch der vollständige Analysekontext in `context.md` gespeichert. Genau diese Differenz besteht zwischen sichtbarer Klärung und gespeichertem Vollvorschlag. Sie ist die in Kapitel 7 beschriebene Lücke zwischen Vorschau und

gespeichertem Stand.

Antworten im Kommentar

Die meldende Person hat die Fragen im selben Issue beantwortet, erneut mit führendem @dev-assist: Screenshot-Unterstützung soll Issue und Kommentare umfassen. Gefundene Bildinhalte sollen im erzeugten Tickettext sichtbar werden.

Strukturierter Vorschlag nach Klärung

Nach der erneuten Analyse entstand ein Vorschlagskommentar. Nach @dev-assist publish übernahm das Issue den gespeicherten Stand. Der veröffentlichte Titel lautete *Use issue screenshots in Dev-Assist analysis*. Die Beschreibung folgte dem kompakten Schema mit Ziel, Akzeptanzkriterien, technischem Kontext und Lösungsvorschlag, wie Abbildung 5.3 zeigt.

Im untersuchten Stand prüft der Publisher nicht, ob genau diese Revision vollständig sichtbar und freigegeben war.

B. Dokumentierter Prüfstand

Die folgende Übersicht wiederholt den in Kapitel 6 dokumentierten Lauf. Die Kurzform soll die Prüfung nachvollziehbar machen.

Tabelle B.1: Kurzprotokoll des dokumentierten Build- und Testlaufs

Größe	Angabe
Datum und Umgebung	31.07.2026, Windows, Node.js v24.11.0, npm 11.6.1
Untersuchter Stand	Git-Commit d179eb22beb0a59006869a3acc7829f3070fc678
Build	npm run build, ohne TypeScript-Fehler
Tests	npm test: 60 von 60 Tests in 15 Suiten bestanden, ca. 0,6 Sekunden
Testabdeckung	nicht konfiguriert
Ende-zu-Ende gegen GitLab	nicht ausgeführt

Die Tests prüfen vor allem Parser, Schema, Formatter, ausgewählte Laufzeitpfade und Aufbereitungslogik. Sie ersetzen weder einen realen Webhook-Lauf noch eine fachliche Bewertung von Modellantworten.

C. Ausgabeschema des Prototyps

Das implementierte Ausgabeschema akzeptiert ausschließlich die folgenden Felder. Unbekannte Felder oder falsche Datentypen werden abgelehnt.

Tabelle C.1: Felder des kompakten Ausgabeschemas

Feld	Typ	Zweck
title	Zeichenkette	prägnanter Issue-Titel
description	Zeichenkettenliste	Ziel, Nutzen, Verhalten und Abgrenzung in Absätzen
acceptanceCriteria	Zeichenkettenliste	beobachtbare Prüfkriterien
technicalContext	Zeichenkettenliste	belegte technische Hinweise, Logs, Annahmen
proposedSolution	Zeichenkettenliste	grobe Umsetzungsschritte ohne fachliche Erfindung
openQuestions	Zeichenkettenliste	unbeantwortete fachliche Fragen

Quellenbasis, Risiken und Validierungsschritte des fachlichen Zielmodells aus Kapitel 3 besitzen im kompakten Ausgabeschema keine eigenen Felder. Sie können nur innerhalb der allgemeinen Listen erscheinen oder entfallen.